

Contenido

PROCESO UNIFICADO DE DESARROLLO (PUD).	4
CONCEPTOS BASICOS.	5
FLUJO DE TRABAJO → Conjunto de actividades referidas al desarrollo de un sistema, referidas a un aspecto de este en particular (requisitos, análisis, diseños) estableciendo los trabajadores que participan, las actividades que se realizan y los artefactos que se producen.	5
TRABAJADOR → Rol de personas responsables de realizar las actividades y producir los artefactos.	5
ACTIVIDAD → Las distintas tareas a realizar en cada flujo de trabajo para realizar los artefactos necesarios.	5
ARTEFACTO → Resultado que se produce o elabora en un flujo de trabajo.	5
ASPECTOS CARACTERISTICOS DEL PUD.	5
EL CICLO DE VIDA DEL PUD.	6
FASES DEL CICLO DE VIDA DEL PUD.	6
1. Fase de inicio o concepción	6
2. Fase de elaboración	6
3. Fase de construcción:	6
4. Fase de transición	6
Modelo de Análisis. Resumen.	7
¿Cuándo ocurre que los objetos se comportan de manera distinta conforme al paso del tiempo?	7
¿Entonces cuándo se necesita un diagrama de máquina de estado?	7
Cuadro conceptual comparativo del modelo de requerimientos y de análisis.	8
Cuadro conceptual comparativo del modelo de CU y de análisis.	8
Describa detalladamente como está compuesto el modelo resultante del workflow de análisis.	9
Modelo de análisis:	9
Clases de análisis:	9
Realizaciones de CU de análisis:	9
Paquetes del análisis:	10
Descripción de la arquitectura (Vista del modelo de análisis):	10
Explique en qué casos recomienda el Proceso Unificado de Desarrollo la realización de un Modelo de Análisis, y que rol tiene el Análisis en el contexto del ciclo de vida iterativo e incremental, que propone el PUD.	10
Explique los tres tipos de clases que se utilizan para modelar el análisis, según el PUD y ejemplifíquelas.	11
Clases de análisis	11
Describa las responsabilidades de los trabajadores involucrados en el Flujo de Trabajo de Análisis y vincúlelos con los artefactos que producen.	11
Explique cuáles son los artefactos de salida del Workflow de Análisis y describa los diagramas de UML 2.0 que se utilizan para construirlos.	12
Explique por qué es necesario considerar el análisis como una etapa diferente a la captura de requisitos o del diseño y la implementación.	13
UML 2.0:	14

¿Cuáles son los diagramas que UML 2.0? propone para crear modelos durante el Workflow de Análisis? Explique el propósito de cada uno, sus características y los conceptos que modela.	14
Mencione y describa los diagramas de UML 2.0 que se clasifican como diagramas de estructura.	14
Especifique como está compuesto el modelo conceptual de UML 2.0, y describa en términos generales los elementos que lo componen.	15
a) Elementos.	15
b) Relaciones	15
c) Diagramas.	15
¿Qué modelan los diagramas de interacción? Realice una clasificación de cada uno de ellos.	16
Explique la diferencia entre los conceptos: Modelo, Vista y Diagrama. Ejemplifique.	16
¿Cuáles son los diagramas de UML 2.0 que modelan objetos (instancias de clases)? Describa cada uno de ellos y mencione en que workflows del Proceso Unificado se los puede utilizar.	17
¿Cuáles son los diagramas que UML 2.0 propone para modelar la estructura del software? Considere los que podemos aplicar en el Flujo de Trabajo de análisis: explique el propósito de cada uno y en qué actividad del F.T. se utilizan.	17
Patrones GRASP:	18
Mencione los patrones GRASP utilizados en el Análisis, describa cuál es su utilidad y desarrolle un ejemplo PROPIO en el que aplique los patrones CREADOR y BAJO ACOPLAMIENTO.	18
¿Para qué se aplican los Patrones GRASP? ¿En qué actividad del F.T. de Análisis se hace uso de los mismos?	19
Completar el siguiente cuadro para que el patrón se corresponda con la afirmación:	19
Para el patrón Experto plantee un ejemplo PROPIO concreto de:	19
¿Cuál es la utilidad de los patrones GRASP? Explique su propósito y desarrolle un ejemplo PROPIO en el que use el patrón CONTROLADOR y un ejemplo en el que no lo use.	20
Resumen de clasificación de PATRONES GRASP.	20
PARCIAL 2: Diseño de Sistemas de Información.	21
¿Qué es el diseño?	21
¿Qué es la arquitectura?	22
¿En qué aspectos deben diseñarse la arquitectura?	22
Workflow de Diseño. Propósito.	23
Descripción del proceso de diseño arquitectónico.	24
MODELADO DE LA ARQUITECTURA.	26
¿PARA QUÉ DOCUMENTAR ARQUITECTURA?	26
SISTEMAS DISTRIBUIDOS. Procesamiento distribuido.	26
ARQUITECTURA MONOLÍTICA.	27
PATRONES ARQ. Vs. ESTILOS ARQUITECTONICOS.	28
PATRON ARQUITECTONICO.	28
ESTILO ARQUITECTONICO.	28
PATRONES ARQUITECTONICOS -----	29
Arquitectura Estratificada o en capas (Layered Style).	29
Arquitectura hexagonal o de puertos y adaptadores.	30

ARQUITECTURA HEXAGONAL VS. LAYERED.	32
CLIENTE/SERVIDOR: VARIANTES. DOS CAPAS.	32
Cliente Ser. N-Tier.	33
Formas destacadas de implementar el patrón C/S:	34
PATRON MAESTRO ESCLAVO o master-slave.	34
Arquitectura Peer-to-Peer.	35
ESTILOS PARA EL DISEÑO DE APLICACIONES WEB.	36
The model-view-controller MVC	36
Service Oriented Architecture SOA.	37
SOA con Service Bus (ESB)	37
SOA DESVENTAJAS.	37
SOA VENTAJAS.	38
Event Driven Architecture.	38
Event Driven Architecture (Con topología Broker).	39
Event Driven Architecture (Con topología Mediator)	39
EDA VENTAJAS.	39
EDA DESVENTAJAS.	40
Variantes de Event Driven Architecture.	40
Publish & Suscribe.	41
Agente (Broker).	41
Arquitectura Comunicando (Messaging).	42
Coordinador de proceso (Process Coordinator).	43
Arquitectura de Microservicios.	43
PRINCIPIOS CLAVE.	43
Ventajas.	44
Desventajas.	44
DIFERENCIAS ENTRE MONOLITICAS Y MICROSERVICIOS.	45
Aspecto arquitectónico	45
Monolíticas	45
Microservicios	45
Implementación.	45
Despliegue.	45
Escalabilidad.	45
Mantenimiento y Administración.	45
Orientación a Negocio.	45
Combinación con servicios externos.	45
CUANDO NO USAR MSA (microservicios)?	45
Service-Based Architecture (Arquitectura basada en Servicios SBA).	46
Variantes de arquitectura en sistemas de microservicios:	46

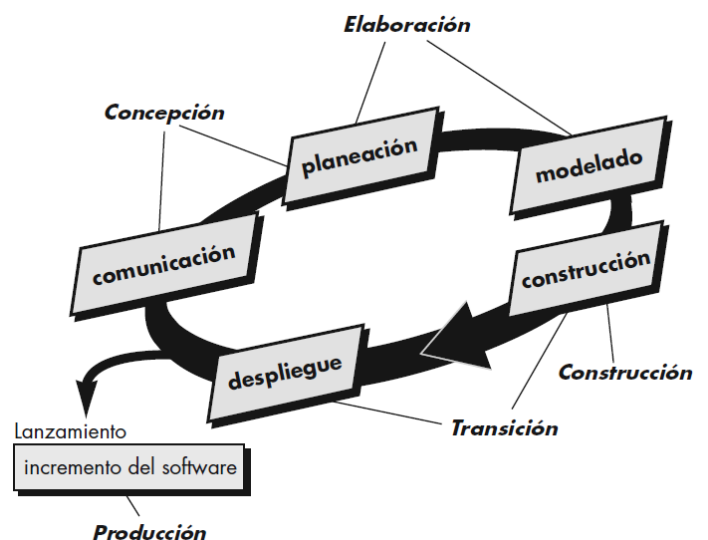
DISEÑO DE PERSISTENCIA (chequear).	47
1. Paradigmas de Diseño de Persistencia	47
2. Modelo Entidad-Relación	47
3. Problema de Impedancia	48
4. Mapeo de Clases a Bases de Datos Relacionales	48
5. Relaciones entre Clases	48
6. Opciones para el Mapeo de Herencia	48
7. Framework de Persistencia	48
8. Beneficios de Usar ODBMS (Sistemas de Gestión de Bases de Datos Orientadas a Objetos)	48
9. Características de las BDOO	48
VISTAS ARQUITECTONICAS.	49
VISTA.	49
MODELO.	49
Vista Arquitectónica de la Funcionalidad (CU).	49
Vista de Diseño (lógica).	50
La Vista de Procesos (Vista de Interacción).	50
La Vista de Implementación (componentes).	51
La Vista de Despliegue.	51
Principios de diseño de software orientado a objetos.	¡Error! Marcador no definido.

PROCESO UNIFICADO DE DESARROLLO (PUD).

El Proceso Unificado de Desarrollo (PUD) es un **marco de trabajo** ampliamente utilizado en el desarrollo de software que *integra las mejores prácticas de diversos métodos y enfoques*. El proceso unificado reconoce la importancia de la **comunicación con el cliente** y los **métodos directos** para describir su punto de vista respecto de un sistema (el **caso de uso**). Hace **énfasis en** la importancia de la **arquitectura del software**. Sugiere un **flujo del proceso iterativo e incremental**, lo que da la sensación evolutiva.

Al principio de la década de 1990, James Rumbaugh, Grady Booch e Ivar Jacobson **crearon un método unificado que combina lo mejor de cada método individual de análisis y diseño orientado a objetos**. Como resultado se obtiene UML, *lenguaje de modelado unificado*, que contiene una notación robusta para el modelado y desarrollo de los sistemas orientados a objetos.

En síntesis:



- ✓ Utiliza el **Lenguaje Unificado de Modelado (UML)** para preparar todos los esquemas de un sistema de software.

UML: Lenguaje estándar para escribir planos de software, que permite visualizar, especificar, construir y documentar artefactos de un sistema.

- ✓ **PUD** establece actividades para crear distintos modelos, utilizando UML para su representación, considerando la filosofía del paradigma orientado a objetos. El UML brinda la tecnología necesaria para apoyar la práctica de la ingeniería de software orientada a objetos, pero no da la estructura del proceso que guíe al equipo del proyecto cuando aplican la tecnología.

CONCEPTOS BASICOS.

FLUJO DE TRABAJO → Conjunto de actividades referidas al desarrollo de un sistema, referidas a un aspecto de este en particular (requisitos, análisis, diseños) estableciendo los trabajadores que participan, las actividades que se realizan y los artefactos que se producen.

TRABAJADOR → Rol de personas responsables de realizar las actividades y producir los artefactos.

ACTIVIDAD → Las distintas tareas a realizar en cada flujo de trabajo para realizar los artefactos necesarios.

ASPECTOS CARACTERISTICOS DEL PUD.

CONDUCIDO POR CASOS DE USO.

Un caso de uso (CIJ) es un fragmento de funcionalidad del sistema que proporciona al usuario un resultado importante, **los casos de uso representan los requerimientos funcionales**.

Permiten especificar los requerimientos de un sistema.

Guían el diseño, implementación y prueba: guían el proceso de desarrollo.

Dirigido por casos de uso quiere decir que el proceso de desarrollo sigue un hilo conductor, es **decir avanza a través de una serie de flujos de trabajo que parten de la identificación de los casos de uso**, los cuales se especifican, diseñan, implementan y prueban.

CENTRADO EN LA ARQUITECTURA

Arquitectura de software: incluye los aspectos estáticos y dinámicos más significativos del sistema. Describe mediante diferentes vistas el sistema en construcción, considerando **diversos factores**, como ser la plataforma en que funcionará el software, dispositivos de hardware, conexiones de red, bloques de construcción, entre otras.

La arquitectura es una vista del diseño completo con las características más importantes resaltadas. Los casos de uso deben encajar en la arquitectura cuando se llevan a cabo. La arquitectura debe permitir el desarrollo de todos los casos de uso requeridos.

CICLO DE VIDA ITERATIVO E INCREMENTAL.

- **Iteraciones** hacen referencia a una serie de pasos en el flujo de trabajo.
- **Incrementos** refieren al crecimiento del producto.

La iteración **trata un grupo de casos de uso que juntos amplían la utilidad del producto desarrollado.**

En cada iteración se realizan actividades de análisis de requisitos, análisis propiamente dicho, diseño, implementación y prueba del sistema.

EL CICLO DE VIDA DEL PUD.

- **El proceso se repite a lo largo de una serie de ciclos que constituyen la vida de un sistema.**
- Cada ciclo produce una nueva versión del sistema para los clientes. **Cada versión es un producto preparado para su entrega.**
- *Producto:* incluye requisitos, casos de uso, especificaciones no funcionales, casos de prueba, modelo de arquitectura, código fuente, componentes ejecutables.
- **Cada ciclo consta de cuatro fases: inicio, elaboración, construcción y transición.**
- Cada fase se subdivide en iteraciones.

FASES DEL CICLO DE VIDA DEL PUD.

El ciclo se divide en **4 fases**, cada fase define **momentos de madurez del producto**, en donde al terminar una de ellas establece la **disponibilidad de un conjunto de artefactos**. Cada fase tiene mínimo **2 o más iteraciones**, siguiendo el concepto de que el **PUD es un desarrollo iterativo e incremental**.

1. Fase de inicio o concepción: Descripción del producto final, análisis del negocio, determinación de las principales funciones y arquitectura del sistema, plan del proyecto y presupuesto.

2. Fase de elaboración: Especificación de casos de uso, diseño de la arquitectura, distintas vistas arquitectónicas. El modelo de análisis desempeña un papel fundamental en la fase de elaboración del ciclo de vida del PUD, donde se enfoca en comprender en detalle los requisitos del sistema y establecer una base sólida para el diseño y la implementación. Antes de comenzar a diseñar e implementar, es necesario contar con una comprensión precisa y detallada de los requerimientos, la cual se consigue en la etapa de análisis.

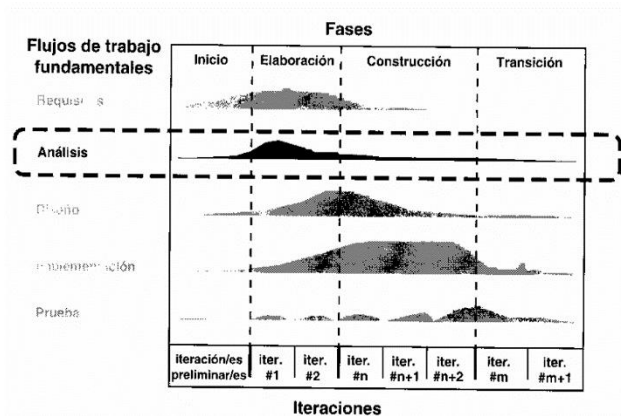
3. Fase de construcción: **Creación del producto, codificación.** Con el objetivo de que reflejen la versión final del incremento del software.

4. Fase de transición: El producto es versión beta. Instalación y prueba del sistema de parte de los usuarios, ajustes según observaciones (entrega y retroalimentación), junto con la información de apoyo (manual de usuario, de instalación, solución de problemas).

Al finalizar esta fase el software se convierte en un producto utilizable que se entrega al cliente.

WORKFLOW - [REQUERIMIENTOS].

- Desarrollar un modelo del sistema que se va a construir.
- Utiliza casos de uso para construir el modelo.
- Los requerimientos funcionales se estructura de manera natural en casos de uso.
- La mayoría de los requerimientos no funcionales pueden tratarse en el contexto de un caso de uso.



WORKFLOW - [ANÁLISIS].

- Permite una especificación más precisa de los requerimientos.
- Se describe usando el lenguaje del desarrollador.
- Estructura los requerimientos.
- Puede verse como un primer corte del modelo de diseño.

Modelo de Análisis.

Resumen.

- **Ofrece una especificación más precisa de los requerimientos** que la que tenemos como resultado de la captura de requerimientos, incluyendo al modelo de casos de uso.
- **Se describe utilizando el lenguaje de los desarrolladores**, y puede por tanto introducir un mayor formalismo y ser utilizado para razonar sobre los funcionamientos internos del sistema,
- Estructura los req. de un modo que facilita su comprensión, su preparación, su modificación, y en general, su mantenimiento.

Un modelo de análisis puede considerarse como una primera aproximación al modelo de diseño (aunque es un modelo por sí mismo), y es por tanto una entrada fundamental cuando se da forma al sistema en el diseño y en la implementación. Esto se debe a que debería ser mantenible el sistema en su conjunto, y no sólo la descripción de sus requisitos.

¿Cuándo ocurre que los objetos se comportan de manera distinta conforme al paso del tiempo?

Los objetos pueden comportarse de manera diferente con el paso del tiempo **cuando necesitan cambiar su estado o su comportamiento en respuesta a eventos o condiciones específicos**. Por ello, se utiliza una máquina de estados para modelar y representar estos cambios, lo que permite una efectiva representación del comportamiento dinámico del sistema a lo largo del tiempo.

¿Entonces cuándo se necesita un diagrama de máquina de estado?

La necesidad de una máquina de estados surge **cuando los objetos en el sistema pueden estar en diferentes estados y necesitan cambiar de un estado a otro en respuesta a eventos externos o internos**. Estos eventos pueden desencadenar transiciones entre estados, y cada estado puede estar asociado con un conjunto particular de comportamientos o acciones que el objeto realiza mientras se encuentra en ese estado.

Cuadro conceptual comparativo del modelo de requerimientos y de análisis.

Aspecto	Modelo de Requerimientos	Modelo de Análisis
Perspectiva	Centrado en las necesidades y expectativas del cliente.	Centrado en la comprensión y modelado del sistema a desarrollar.
Propósito	Identificar y definir los objetivos y funcionalidades del sistema.	Descomponer y analizar las funcionalidades en detalle.
Representación	Utiliza técnicas como historias de usuario, casos de uso, etc.	Emplea diagramas de clases, 7 diagramas de comportamiento
Enfoque de abstracción	Orientado a describir qué debe hacer el sistema.	Orientado a cómo el sistema debe hacerlo y cómo se estructura.
Identificación de Actores	Identifica los usuarios y otros sistemas externos.	Analiza cómo interactúan los actores y objetos del sistema.
Funcionalidades	Describe las funcionalidades de alto nivel del sistema.	Descompone las funcionalidades en acciones más específicas.
Especificación de Requisitos	Se enfoca en los requisitos del sistema desde el punto de vista del usuario.	Detalla los requisitos funcionales y no funcionales del sistema.
Iteratividad	Puede ser iterativo para refinar y validar los requisitos.	Puede iterar para ajustar y mejorar el diseño del sistema.
Nivel de Detalle	A menudo se presenta en un nivel menos detallado y general.	Se detalla más para comprender en profundidad el sistema.

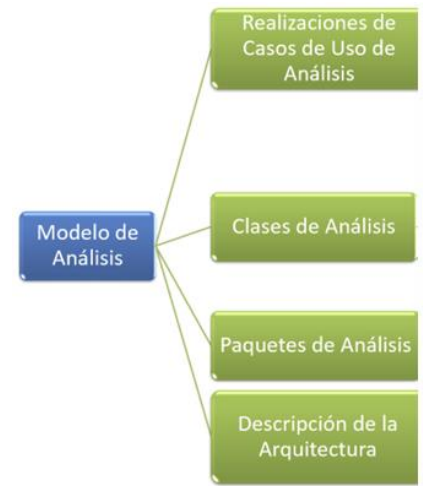
Cuadro conceptual comparativo del modelo de CU y de análisis.

Modelo de CU	Modelo de Análisis
Usa el lenguaje del cliente.	Usa el lenguaje del desarrollador.
Vista externa del sistema.	Vista interna del sistema.
Usado como contrato entre clientes y desarrolladores, para decidir que debe (y no) hacer el sistema.	Usado por los desarrolladores para saber cómo darle forma al sistema.
Estructurado por CU.	Estructurado por clases y paquetes.
Captura la funcionalidad del sistema.	Esboza como llevar a cabo la funcionalidad del sistema.
Puede contener redundancias, inconsistencias entre requerimientos.	No debe contener redundancias o inconsistencias entre requerimientos.
Define CU que serán analizados en el mod. Análisis.	Define realizaciones de CU y c/u de ellas representa el análisis de un caso de uso del modelo de casos de uso.

Describe detalladamente como está compuesto el modelo resultante del workflow de análisis.

El modelo resultante del workflow de análisis parte del:

Modelo de análisis: Modelo de objetos conceptual que nos ayuda a refinar y estructurar los requerimientos, nos permite razonar sobre **aspectos internos del sistema** y nos **proporciona una estructura centrada en el mantenimiento**. Esta puede servir como entrada para las actividades de diseño e implementación, por ello puede considerarse como *una primera aproximación al diseño*.



- El modelo hace abstracciones y evita resolver algunos problemas y tratar requerimientos que es mejor posponer al diseño e implementación (**RNF**).

Clases de análisis:

- Representa una abstracción de una o varias clases/subsistemas del modelo de dominio.
- Se **centra en el tratamiento de requerimientos funcionales** y pospone los no funcionales.
- Su **comportamiento se define mediante responsabilidades** en un alto nivel.
 - **Una responsabilidad es una descripción textual de un conjunto cohesivo del comportamiento de una clase.**
- Define atributos de alto nivel.
- Participa en relaciones, aunque más conceptuales que las de diseño.

Puede ser de alguno de los 3 tipos, estandarizados en UML, que ayuda a los desarrolladores a distinguir una de otra. Estas son:

1. **Clase de interfaz:** Modela las **interacciones entre un sistema y sus actores**. Estas a menudo implican presentar (y recibir) información (y peticiones) desde y hacia los usuarios y sistemas externos.
2. **Clase de entidad:** Modela el **comportamiento y la información asociada a algún fenómeno o concepto**, que posee una vida larga y a menudo es consistente.
Suelen derivarse de clases de entidad del negocio (o del dominio del problema).
3. **Clase de control:** Representa coordinación, secuencia, transacciones y control de otros objetos, y **se usa para encapsular el control de un caso de uso en concreto**.
También representa cálculos y derivaciones complejas que no pueden asociarse a ninguna clase de entidad concreta.
 - ✓ Modela los **aspectos dinámicos del sistema**.
 - ✓ Son el **nexo entre las clases de entidad y de interfaz**, ya que **no pueden relacionarse directamente**.

Realizaciones de CU de análisis: Representan una **colaboración dentro del modelo de análisis y describe como se lleva a cabo la realización de un CU en concreto**, en términos de las clases de análisis y sus objetos en **interacción, centrándose** de manera natural en los **requisitos funcionales**.

- Posee una **descripción textual del flujo de sucesos**, un diagrama de clases que muestra sus clases de análisis participantes y un diagrama de interacción que muestra la realización de un escenario particular del CU, en término de las interacciones entre objetos de dichas clases.

Paquetes del análisis: Proporcionan **un medio para organizar los artefactos del análisis en piezas manejables**.

- Pueden estar compuesto de clases de análisis, realizaciones de CU de análisis y otros paquetes.
- Pueden representar una separación de intereses de análisis, para subdividir el conocimiento del dominio y ser analizados de manera concurrente.
- Deben ser **altamente cohesivos** (*su contenido debe estar fuertemente relacionado*) y **débilmente acoplados** (*debe existir una mínima dependencia entre unos y otros*).

Descripción de la arquitectura (Vista del modelo de análisis): Contiene una **vista de la arquitectura del modelo de análisis**, que **muestra** sus **artefactos** más significativos de esta:

1. Clases de análisis fundamentales.
2. Realizaciones de CU que describen cierta funcionalidad crítica.
3. Descomposición del modelo de análisis en paquetes del análisis y sus dependencias.

Explique en qué casos recomienda el Proceso Unificado de Desarrollo la realización de un Modelo de Análisis, y que rol tiene el Análisis en el contexto del ciclo de vida iterativo e incremental, que propone el PUD.

Existen **tres enfoques básicos para el uso del modelo de análisis**:

1. **Mantenimiento continuo del modelo de análisis a lo largo de todo el proyecto**, obteniendo beneficios como la claridad y consistencia en cada iteración.
2. Considerar **el modelo de análisis como una herramienta transitoria**, enfocándose en él durante la fase de elaboración y dejando de actualizarlo en fases posteriores cuando el diseño y la implementación están en curso.
3. **No utilizar el modelo de análisis en absoluto**, integrando los requisitos directamente en la captura de requisitos o en el diseño, lo cual puede ser justificable en casos de requisitos simples o bien conocidos.

Al elegir entre estas variantes, es crucial realizar un análisis de coste/beneficio para determinar si es beneficioso mantener o dejar de actualizar el modelo de análisis a lo largo del proyecto. Aunque evitar el uso del modelo de análisis puede ahorrar costos, generalmente sus ventajas superan los costos asociados, por lo que se recomienda su utilización, a menos que el sistema sea excepcionalmente simple.

El **modelo de análisis desempeña un papel fundamental en la fase de elaboración** del ciclo de vida del PUD, donde se enfoca en comprender en detalle los requisitos del sistema y establecer una base sólida para el diseño y la implementación.

Durante la etapa de elaboración en el Proceso Unificado de Desarrollo (PUD), **las iteraciones iniciales se enfocan en el análisis**, lo que contribuye a **establecer una arquitectura sólida** y a **comprender a fondo los requerimientos del sistema**. En esta fase, el modelo de análisis se utiliza para documentar y representar los artefactos obtenidos del proceso de análisis, colaborando al mantenimiento y la consistencia durante el ciclo de vida del software, que agrega como ventajas:

- ✓ **Identificación temprana de problemas:** Un modelo de análisis sólido permite identificar y corregir errores, inconsistencias o malas prácticas en una etapa temprana del proceso de desarrollo.
- ✓ **Reducción de costos:** Al corregir los problemas en una etapa temprana, se evita el gasto adicional asociado con la reestructuración y modificación del código ya implementado.
- ✓ **Evitar problemas de escalabilidad:** Las inconsistencias o malas prácticas no corregidas durante la etapa de análisis pueden conducir a problemas de escalabilidad a medida que el sistema crece en complejidad y tamaño.

Explique los tres tipos de clases que se utilizan para modelar el análisis, según el PUD y ejemplifíquelas.

Clases de análisis son aquellas que se centran en el **tratamiento de requisitos funcionales**, lo cual es más evidente en el contexto del dominio del problema. Son clases que **tienden a ser más conceptuales**, ya que su comportamiento se define mediante **responsabilidades** en un nivel más alto y menos formal. Estas clases también **definen atributos a un nivel alto**, siendo conceptuales y reconocibles en el dominio del problema

Clase de interfaz: Modela las **interacciones entre el sistema y los actores**. Esta interacción implica recibir (y presentar) información y peticiones de (y hacia) los usuarios y sistemas externos.

Clase de entidad: Modela la **información y el comportamiento asociado a algún fenómeno o concepto que posee vida larga y** que, a menudo, es **persistente**. Por lo general, derivan de clases de entidad del negocio (o del dominio del problema), reflejan información beneficiosa para los desarrolladores al diseñar e implementar dicha funcionalidad.

La **diferencia de clase de entidad respecto a clases de entidad del negocio** (del dominio), es que las clases del dominio describen el contexto del sistema y por lo tanto pueden incluir información que el sistema no maneja en absoluto.

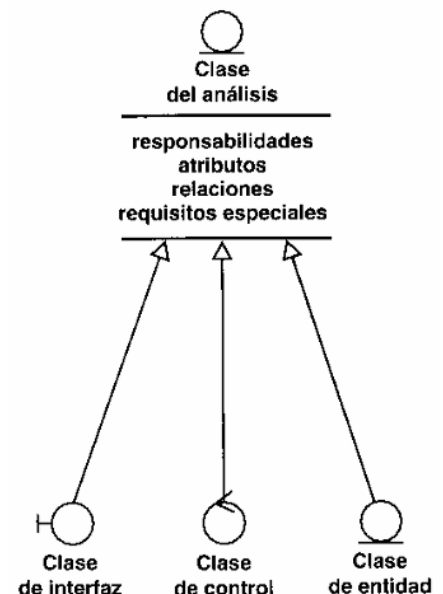
Clase de control: Representan coordinación, secuencia, transacciones y control de otros objetos, y se usan para encapsular el control de un CU en concreto. También representan cálculos y derivaciones complejas (lógica de negocio) que no pueden asociarse a ninguna clase de entidad concreta.

- Modela los aspectos dinámicos del sistema, manejan y coordinan las acciones y los flujos de control principales del CU, delegando trabajo a otros objetos.
- Son el nexo entre las clases de entidad e interfaz.

Describa las responsabilidades de los trabajadores involucrados en el Flujo de Trabajo de Análisis y vincúlelos con los artefactos que producen.

En el workflow de análisis, podemos encontrar 3 trabajadores:

Arquitecto: Es responsable de la integridad del **modelo de análisis**, garantizando que este sea correcto, consistente y legible como un todo. El modelo es correcto siempre y cuando cumpla con la funcionalidad descrita en el modelo de CU, y solo esa.



- ✓ También es responsable de la **arquitectura**, es decir, de la existencia de las partes significativas de esta.

Ingeniero de CU: Responsable de mantener la integridad de una o más **realizaciones de CU**, garantizando que cumplan los requerimientos que recaen sobre ellas. No es responsable de las clases de análisis ni de las relaciones que se usan en la realización del caso de uso.

Ingeniero de Componentes: Responsable de definir y mantener atributos, responsabilidades, relaciones y requerimientos especiales de una o más **clases de análisis**, garantizando que cumplan con los requerimientos que se esperan de ella.

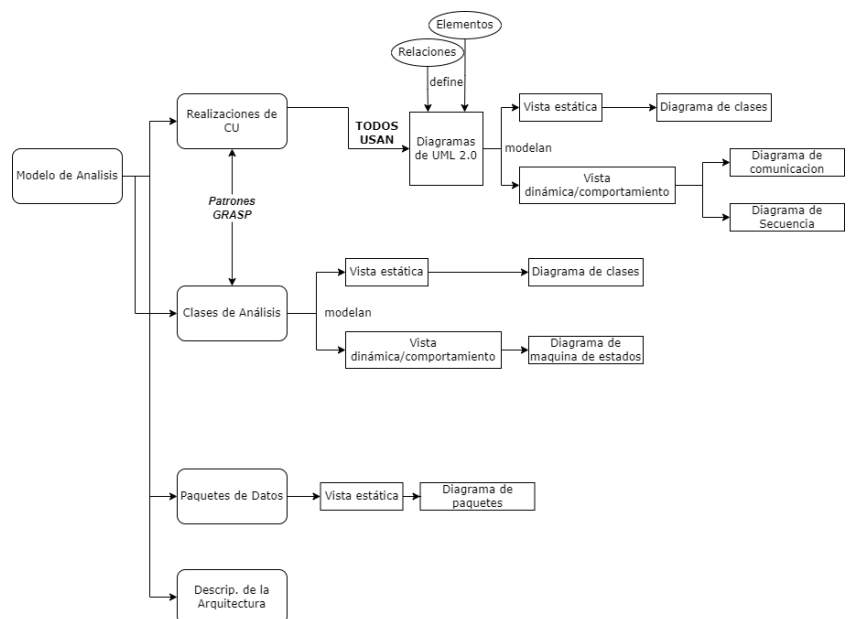
- ✓ También es responsable de la integridad de los **paquetes del análisis**, garantizando que su contenido sea correcto.

Explique cuáles son los artefactos de salida del Workflow de Análisis y describa los diagramas de UML 2.0 que se utilizan para construirlos.

Los artefactos de salida del workflow de análisis son:

Modelo de análisis: Modelo de objetos conceptual, que nos ayuda a refinar y estructurar los requerimientos, nos permite razonar sobre aspectos internos del sistema, y nos proporciona una estructura centrada en el mantenimiento, en aspectos tales como la flexibilidad ante cambios y la reutilización.

Clases de análisis: Abstracción que modela un elemento específico del dominio del problema.



- Se centra en el tratamiento de requerimientos funcionales y pospone los no funcionales.
- Define atributos en un alto nivel.
- Participa en relaciones, más conceptuales que las de diseño e implementación.
- Su comportamiento se define mediante responsabilidades en un alto nivel.

Pueden ser de 3 tipos distintos, los cuales están estandarizados en UML y ayudan a los desarrolladores a distinguirlas:

1. **Clases de interfaz:** Modela las interacciones entre el sistema y sus actores. Estas consisten, a menudo, en presentar (y recibir) información (y peticiones) desde (y hacia) los usuarios y sistemas externos.
 2. **Clases de entidad:** Modela la información y el comportamiento asociado a algún fenómeno o concepto, que poseen vida larga y, a menudo, son persistentes.
- Por lo general, se derivan de clases de entidad del negocio (o del dominio del problema).
 - 3. **Clases de control:** Representan coordinación, secuencia, transacciones y control de otros objetos, y se usan para encapsular el control de un CU en concreto.
 - También representan derivaciones y cálculos complejos que no pueden asociarse a ninguna clase de entidad concreta. Modela los aspectos dinámicos del sistema. Son el nexo entre las clases de interfaz y entidad.

Realizaciones de CU de análisis: Representan una colaboración dentro del modelo de análisis, que describe como se lleva a cabo y ejecuta un caso de uso determinado, en término de las clases de análisis participantes y los objetos de estas que interactúan.

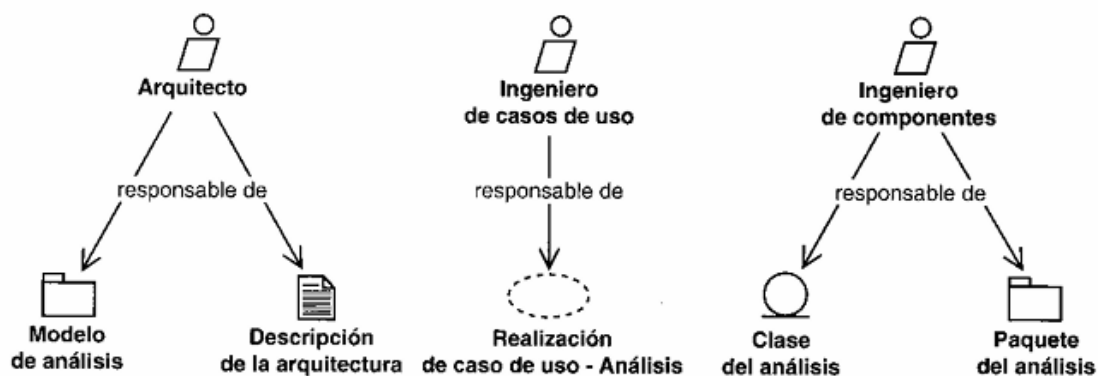
- Poseen una descripción textual del flujo de sucesos, un diagrama de clases que muestra las clases de análisis participantes, y un diagrama de interacción que muestra la realización de un escenario particular del caso de uso.

Paquetes del análisis: Proporcionan un medio para organizar los artefactos del modelo de análisis en piezas manejables. Estos pueden contener clases de análisis, realizaciones de CU de análisis y otros paquetes (recursivamente).

- Los paquetes deben ser altamente cohesivos (su contenido debe estar fuertemente integrado) y débilmente acoplados (deben depender mínimamente unos de otros).

Descripción de la arquitectura (Vista del modelo de análisis): Contiene una vista de la arquitectura del modelo de análisis, que muestra sus artefactos más significativos para la arquitectura. Estos son:

- La descomposición del modelo de análisis en paquetes de análisis y sus dependencias.
- Clases de análisis fundamentales.
- Realizaciones de CU de análisis que describan una funcionalidad importante y crítica.



Trabajadores y artefactos implicados en el análisis.

Explique por qué es necesario considerar el análisis como una etapa diferente a la captura de requisitos o del diseño y la implementación.

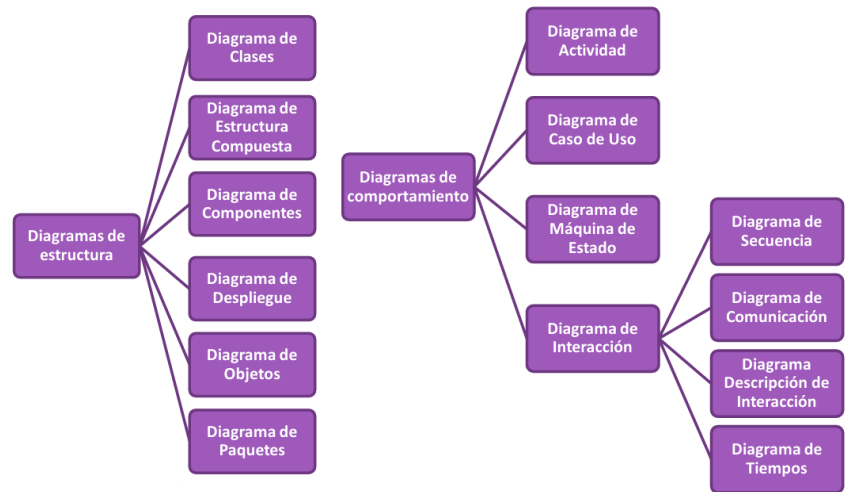
Se dice que **el análisis no es diseño ni implementación**, debido a que estas 2 etapas son mucho más que el análisis. Dichas etapas, se preocupan de dar forma al sistema, de manera que de vida a los requerimientos que incorpora, considerando atender los requerimientos de calidad(RNF), los cuales definirían los límites más tangibles del sistema, como su performance, seguridad, compatibilidad, etc.

Antes de comenzar a diseñar e implementar, es necesario contar con una **comprensión precisa y detallada de los requerimientos funcionales, la cual se consigue en la etapa de análisis.**

Por ello, se dice que con el análisis se consigue una **separación de intereses** que prepara y simplifica las actividades subsiguientes, delimitando los temas que deben resolverse y las decisiones a tomarse en dichas actividades.

UML 2.0:

¿Cuáles son los diagramas que UML 2.0 propone para crear modelos durante el Workflow de Análisis? Explique el propósito de cada uno, sus características y los conceptos que modela.



Los diagramas que UML 2.0 propone para crear modelos, durante el workflow de análisis, son:

- Diagrama de clases:** Muestran un conjunto de clases, interfaces y colaboraciones, así como sus relaciones. Modelan la parte estática del sistema.
- Diagrama de paquetes:** Muestra la descomposición del modelo de análisis en unidades organizativas, y sus dependencias.
- Diagrama de estados:** Muestra una máquina de estados, con sus respectivos estados, transiciones, eventos y actividades.
- Diagrama de comunicación:** Modela un conjunto de objetos, incluyendo los mensajes que intercambian entre sí. Modelan la parte dinámica del sistema. Resaltan la organización estructural de dichos objetos.
- Diagrama de secuencia:** Ídem al de comunicación, con la diferencia que este resalta la ordenación temporal de los mensajes.
- Diagrama de timing:** Fusiona los diagramas de secuencia y de estado, y muestra los estados de un objeto a lo largo del tiempo, como así también los mensajes que afectan a este.
- Diagrama de descripción de la interacción:** Fusiona los diagramas de secuencia y de actividad, para permitir que los fragmentos de interacción sean fácilmente combinados con los puntos de decisión. Modelan la parte dinámica del sistema.

Mencione y describa los diagramas de UML 2.0 que se clasifican como diagramas de estructura.

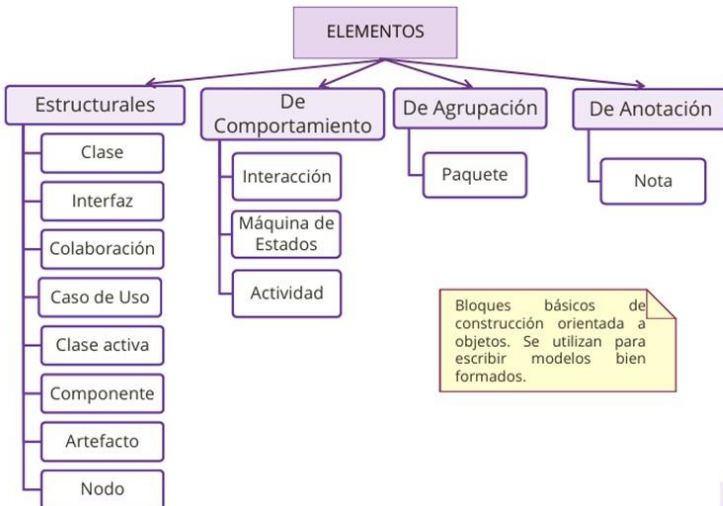
En UML 2.0, existen **13 diagramas**, **6** de los cuales se denominan **diagramas de estructura**. Estos son:

- Diagrama de clases:** Muestra un conjunto de clases, interfaces y colaboraciones, así como sus relaciones.
- Diagrama de objetos:** Muestra un conjunto de objetos, así como sus relaciones.
- Diagrama de paquetes:** Muestra la descomposición de un paquete en piezas manejables y sus dependencias.
- Diagrama de estructura compuesta:** Provee un medio para ordenar una estructura de elementos, y focalizar en detalles, construcción y relaciones internas.
- Diagrama de componentes:** Modela estructuras de alto nivel, construidas desde una o más clases, y provee una interfaz bien definida.

6. **Diagrama de despliegue:** Muestra la configuración de nodos de procesamiento en tiempo de ejecución, y los artefactos que residen en ellos.

Especifique como está compuesto el modelo conceptual de UML 2.0, y describa en términos generales los elementos que lo componen.

El modelo conceptual de UML 2.0 está compuesto por 3 elementos principales:



1) Bloques básicos de construcción.

a) Elementos.

✓ **Estructurales:** Partes **estáticas** de un modelo. Pueden ser:

- **Clase.** Abstracción de objetos
- **CU.** Interacción funcional
- **Interfaz.** Punto de interacción
- **Colaboración.** Trabajo conjunto entre objetos
- **Clase activa.** Clase con comportamiento
- **Componente.** Bloque de construcción
- **Artefacto.** Producto del desarrollo

- **Nodo.** Punto de conexión

✓ **De comportamiento:** Partes **dinámicas** de un modelo. Pueden ser:

- **Interacción.** Intercambio dinámico de mensajes
- **Máquina de estados.** Representación del comportamiento
- **Actividad.** Proceso o flujo

✓ **De agrupación:** Partes **organizativas** de un modelo. Pueden ser:

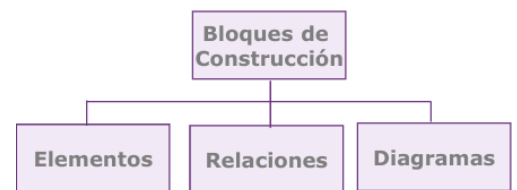
- **Paquetes.** Organización de elementos relacionados

✓ **De anotación:** Partes **explicativas** de un modelo. Pueden ser:

- **Notas.** Información adicional

b) Relaciones.

- ✓ **Asociación.** Representa una relación entre dos clases.
- ✓ **Agregación.** Relación "todo-parte" entre clases, donde una clase contiene instancias de otra.
- ✓ **Generalización.** Herencia de clases
- ✓ **Realización.** Implementación de interfaces



c) Diagramas.

- ✓ De **estructura** (6).
- ✓ De **comportamiento** (7).

2) **Reglas** que dictan como combinar estos bloques. Estas son reglas sintácticas y semánticas para:

- a) Nombres.
- b) Alcance.
- c) Visibilidad.
- d) Integridad.

e) Ejecución.

3) **Mecanismos comunes** que se aplican a través de UML. Estos son:

- a) Especificaciones.
- b) Adornos.
- c) Divisiones Comunes.
- d) Mecanismos de extensibilidad:
 - ✓ Estereotipos.
 - ✓ Valores etiquetados.
 - ✓ Restricciones.



¿Qué modelan los diagramas de interacción?

Realice una clasificación de cada uno de ellos.

Los diagramas de interacción modelan un conjunto de objetos que se comunican intercambiando mensajes entre sí. Existen **4 tipos**:

- ✓ **Diagrama de secuencia:** Resalta la ordenación temporal de los mensajes.
- ✓ **Diagrama de comunicación:** Resalta la organización estructural de los objetos que se comunican.
- ✓ **Diagrama de timing:** Combina los diagramas de secuencia y de estados, para proveer una vista del estado de un objeto a través del tiempo, y los mensajes que lo modifican.
- ✓ **Diagrama de descripción de la interacción:** Fusiona los diagramas de secuencia y de actividad, para permitir que los fragmentos de interacción sean fácilmente combinados con los puntos y flujos de decisión.

Explique la diferencia entre los conceptos: Modelo, Vista y Diagrama.

Ejemplifique.

Modelo: Es la representación simplificada de un sistema, elaborada para predecir, controlar y comprender el funcionamiento del sistema. Por ejemplo, se tiene el modelo de requerimientos para comprender y estudiar el dominio del problema.

Ejemplo: En el contexto del desarrollo de software, un modelo puede ser una representación conceptual de los requisitos, la arquitectura o el diseño de un sistema. Por ejemplo, un modelo de casos de uso representa las interacciones entre un sistema y sus usuarios, pero no describe detalles de implementación.

Vista: Es una perspectiva del sistema. Las vistas son 5: casos de uso; diseño; interacción; implementación y despliegue; y se ilustran a través de diagramas. Un ejemplo es una vista de diseño, que se ilustra estáticamente mediante un diagrama de clases, para comprender las clases y relaciones que forman el vocabulario del problema y la solución.

Diagrama: Es la representación de una perspectiva del sistema que se **ilustra la** mayoría de las veces como un grafo conexo de nodos y arcos. Permite visualizar partes del sistema e ilustrar como estas funcionan. Un ejemplo de diagrama es un diagrama de clases. En el cual se ilustran las clases participantes con sus **atributos, métodos** y relaciones.

¿Cuáles son los diagramas de UML 2.0 que modelan objetos (instancias de clases)? Describa cada uno de ellos y mencione en que workflows del Proceso Unificado se los puede utilizar.

Los diagramas de UML 2.0 usados para modelar objetos son:

- **Diagrama de objetos:** Muestran un conjunto de objetos, así también como sus relaciones.
- **Diagrama de interacción:** Modelan un conjunto de objetos y los mensajes que se intercambian entre sí:
- **Diagrama de secuencia:** Enfatiza en la ordenación temporal de los mensajes. Resalta el orden de los mensajes en función del tiempo. Muestra un conjunto de objetos y los mensajes enviados y recibidos por esos objetos.
- **Diagrama de comunicación:** Resalta la organización estructural de los objetos que se comunican. Provee una vista de la colaboración de objetos en un ambiente de tiempo real, distribuyendo funcionalidad en las clases, analizando los aspectos comportamentales de un sistema.
- **Diagrama de timing:** Combina los diagramas de secuencia y de estados, y muestra los estados por los que pasa un objeto a través del tiempo, así también como los mensajes que lo modifican. También muestra la interacción entre eventos dependientes del tiempo y las restricciones de tiempo y duración que los gobiernan.
- **Diagrama de descripción de la interacción:** Fusiona los diagramas de secuencia y de actividad, para permitir que los fragmentos de interacción sean fácilmente combinados con los nodos y flujos de decisión.

¿Cuáles son los diagramas que UML 2.0 propone para modelar la estructura del software? Considere los que podemos aplicar en el Flujo de Trabajo de análisis: explique el propósito de cada uno y en qué actividad del F.T. se utilizan.

En el Flujo de Trabajo (FT) de análisis se suelen utilizar principalmente tres tipos de diagramas: El diagrama de clases, de objetos y de paquetes.

DIAGRAMA DE CLASE.

- El diagrama de Clase muestra los bloques de construcción de cualquier sistema orientado a objetos.
- Los diagramas de clases describen la vista estática del modelo o parte del modelo, describiendo que atributos y comportamientos tienen en lugar de detallar los métodos para realizar operaciones.
- Los diagramas de Clase son más útiles para ilustrar relaciones entre clases e interfaces. Las generalizaciones, agregaciones, y asociaciones son todas valiosas al reflejar herencias, composición o uso, y conexiones respectivamente.
- Actividad del FT: Se utiliza principalmente en la actividad de modelado conceptual y análisis de requisitos. Ayuda a identificar las entidades clave del sistema, sus relaciones y comportamientos básicos.

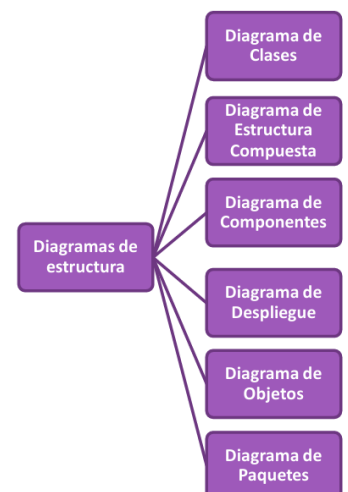


DIAGRAMA DE OBJETOS.

- Un diagrama de objetos puede ser considerado un caso especial de un diagrama de clase.

- Los diagramas de objetos utilizan un subconjunto de los elementos de un diagrama de clases con el fin de enfatizar la relación entre las instancias de clases en algún momento en el tiempo. Son útiles en la comprensión de diagramas de clases.
- No muestran nada diferentes en arquitectura a los diagramas de clases, sino que reflejan la multiplicidad y roles.
- Actividad del FT: Se utiliza para explorar y entender cómo interactúan los objetos del sistema en escenarios específicos. Puede ayudar a identificar patrones de comportamiento y aclarar aspectos del diseño.

DIAGRAMA DE PAQUETES.

- Los diagramas de paquetes se usan para reflejar la organización de paquetes y sus elementos.
- Los usos más comunes para los diagramas de paquete son para organizar diagramas de casos de uso y diagramas de clase, a pesar de que el uso de los diagramas de paquete no es limitado a estos elementos UML.
- Actividad del FT: Se utiliza en la actividad de modelado arquitectónico y diseño del sistema. Ayuda a organizar y visualizar la estructura del sistema en términos de módulos o componentes, lo que facilita la comprensión y comunicación entre los miembros del equipo.

Patrones GRASP:

Mencione los patrones GRASP utilizados en el Análisis, describa cuál es su utilidad y desarrolle un ejemplo PROPIO en el que aplique los patrones CREADOR y BAJO ACOPLAMIENTO.

Los patrones GRASP usados durante el análisis son:

- Experto en información:** Asigna responsabilidades al experto en información, es decir, a la clase que cuenta con la información necesaria para realizar la responsabilidad.
- Creador:** Asigna a la clase A la responsabilidad de crear una instancia de la clase B, si se cumple alguno de estos casos:
 - **A** está formado por **B**.
 - **A** contiene a **B**.
 - **A** está compuesto por **B**.
 - **A** tiene información para crear a **B**.
 - **A** utiliza a **B**.
- Bajo Acoplamiento:** Asigna responsabilidades a un elemento manteniendo un bajo nivel de acoplamiento.
 - El acoplamiento es una medida de la fuerza con que un elemento está conectado, tiene conocimientos o confía en otros elementos.
 - Un elemento débilmente acoplado, es aquel que no depende demasiado de otros elementos.
- Alta Cohesión:** Asigna responsabilidades a un elemento, manteniendo un alto nivel de cohesión.
 - La cohesión mide que tan relacionadas están las responsabilidades de un elemento.
 - Un elemento con alta cohesión posee responsabilidades altamente relacionadas, y no hace una gran cantidad de trabajo.
- Controlador:** Asigna responsabilidades de recibir o manejar un evento del sistema, a una clase que puede representar:

- El sistema global.
- Un escenario de CU donde tiene lugar el evento del sistema.
- Sirve como intermediario entre la interfaz y el algoritmo que la implementa, recibiendo datos del usuario y enviándolos a las distintas clases, dependiendo del método llamado.
- Básicamente, delega en otros objetos el trabajo que necesita hacer, coordina y controla la actividad.

¿Para qué se aplican los Patrones GRASP? ¿En qué actividad del F.T. de Análisis se hace uso de los mismos?

- Un **patrón es una descripción del problema y su solución**, que posee un nombre, se puede **aplicar en diferentes contextos** e incluye consejos de cómo aplicarlos.
- Pretenden codificar **conocimientos**, estilos y principios existentes que **ya han sido probados que son válidos**.
- Los patrones GRASP **describen los principios fundamentales del diseño de objetos**, y se aplican para la asignación de responsabilidades.

En el Framework de Trabajo (FT) de Análisis, los Patrones GRASP se utilizan especialmente en la actividad de definición y análisis de requerimientos. Durante esta actividad, se aplican los Patrones GRASP para comprender y modelar las responsabilidades de las diferentes clases y objetos del sistema, lo que ayuda a establecer una arquitectura sólida y coherente, promoviendo buenas prácticas y claridad del sistema en sí.

Completar el siguiente cuadro para que el patrón se corresponda con la afirmación:

Característica del patrón	Patrón GRASP
Asigna la responsabilidad a la clase que cuenta con la información necesaria para realizar la responsabilidad.	Experto
Asigna la responsabilidad a un elemento, de recibir o manejar un evento del sistema.	Controlador
Asigna la responsabilidad al elemento A, de crear una instancia del elemento B.	Creador
Asigna la responsabilidad a un elemento, de tal manera que sus responsabilidades estén altamente relacionadas entre sí.	Alta Cohesión
Asigna la responsabilidad a un elemento, de modo tal que este no dependa demasiado de otros elementos.	Bajo Acoplamiento

Para el patrón Experto plantee un ejemplo PROPIO concreto de:

a) Interacción que respeta la aplicación del patrón Experto:

Supongamos que tenemos un sistema de gestión de una biblioteca. Para calcular la multa por el retraso en la devolución de un libro, creamos una clase llamada GestorMulta que tiene la responsabilidad de calcular la multa basada en la fecha de devolución y la fecha límite. Esta clase es un experto en la lógica de cálculo de multas porque tiene toda la información necesaria para realizar este cálculo de manera precisa.

b) Interacción que no respeta la aplicación del patrón Experto:

En el mismo sistema de gestión de la biblioteca, podríamos tener una clase llamada Libro que tiene la responsabilidad de calcular la multa por el retraso en la devolución. Sin embargo, esto violaría el principio de experto, ya que la clase Libro debería estar más centrada en representar la información relacionada con el libro (título, autor, ISBN, etc.) y no debería tener la responsabilidad de calcular multas. Esto rompería la cohesión de la clase Libro y crearía acoplamiento innecesario entre diferentes aspectos del sistema.

¿Cuál es la utilidad de los patrones GRASP? Explique su propósito y desarrolle un ejemplo PROPIO en el que use el patrón CONTROLADOR y un ejemplo en el que no lo use.

Los patrones GRASP describen los principios fundamentales del diseño de objetos, y la asignación de responsabilidades, expresados como patrones.

El patrón CONTROLADOR se utiliza para asignar la responsabilidad del manejo de un mensaje de los eventos del sistema a una clase controladora. **Su propósito es garantizar que los procesos del dominio sean manejados por la capa del dominio y no por la de interfaz**, lo que facilita la modularidad y el mantenimiento del sistema.

Ejemplo con el patrón CONTROLADOR:

Imagina un sistema de gestión de biblioteca. Cuando un usuario realiza una solicitud de préstamo de un libro a través de la interfaz de usuario, el controlador de préstamos se encarga de procesar esta solicitud. Este controlador valida la disponibilidad del libro, verifica el estado del usuario y actualiza la base de datos con la información del préstamo. Al separar esta lógica de procesamiento de la interfaz de usuario, el sistema se vuelve más modular y fácil de mantener.

Ejemplo sin utilizar el patrón CONTROLADOR:

En el mismo sistema de gestión de biblioteca, si no se aplica el patrón CONTROLADOR, es posible que la lógica de procesamiento de la solicitud de préstamo se implemente directamente en la interfaz de usuario. Esto podría resultar en una interfaz de usuario más compleja y difícil de mantener, ya que la lógica de negocio estaría entrelazada con la lógica de presentación. Además, cualquier cambio en la forma en que se manejan las solicitudes de préstamo requeriría modificaciones en la interfaz de usuario, lo que aumentaría la fragilidad y la complejidad del sistema.

Resumen de clasificación de PATRONES GRASP.

Patrón	Problema	Solución	Explicación	Beneficios
Controlador	¿Quién debería encargarse de atender un evento del sistema?	Asignar la responsabilidad del manejo de un mensaje de los eventos de un sistema, a una clase controladora.	Un controlador de caso de uso es una buena alternativa cuando hay muchos eventos del sistema entre varios procesos.	Garantiza que los procesos del dominio sean manejados por la capa del dominio y no por la de interfaz. Reflexiona sobre el estado de un caso de uso.
Experto	¿Cuál es el principio fundamental en virtud del cual se asignan las responsabilidades?	Asignar una responsabilidad al experto en información.	Los objetos hacen cosas relacionadas con la información que poseen. Puede haber expertos parciales.	Bajo acoplamiento. El comportamiento se distribuye entre las clases que cuentan con la información requerida.
Patrón	Problema	Solución	Explicación	Beneficios

Creador	¿Quién debería ser el responsable de crear una nueva instancia de alguna clase?	Asignar a la clase B la responsabilidad de crear una instancia de la clase A.	Encontrar un creador que debemos conectar con el objeto producido, esto soporta el bajo acoplamiento.	Ayuda al bajo acoplamiento, supone menos dependencias. La clase creada tiende a ser visible al creador.
Bajo Acoplamiento	¿Cómo dar soporte a una dependencia escasa y a un aumento de reutilización?	Asignar una responsabilidad para mantener bajo el acoplamiento.	Clases más independientes, que reducen el impacto de los cambios, y también más reutilizables.	No se afectan componentes por cambios de otros componentes. Fáciles de entender y reutilizar.
Alta Cohesión	Las clases con baja cohesión a menudo representan un grado de abstracción alto...	Asignar una responsabilidad de modo que la cohesión siga siendo alta.	Una clase de alta cohesión tiene un número relativamente pequeño de operaciones con funcionalidad relacionada.	Mejora la claridad y la facilidad de comprensión. Simplifica la evolución. A menudo genera bajo acoplamiento. Mayor capacidad de reutilización.

PARCIAL 2: Diseño de Sistemas de Información.

“The goal of software architecture is to minimize the human resources required to build and maintain the required system.”

¿Qué es el diseño?

El diseño es el proceso mediante el cual se aplican varias técnicas y principios con el objetivo de definir un dispositivo, un proceso o un sistema con suficiente nivel de detalle como para permitir su realización física.

Proceso iterativo de transformar un modelo lógico físico, teniendo en cuenta las restricciones del negocio.

Se diferencia del modelo de análisis a través de la salida, ya que el **análisis modela la solución en términos lógicos**, mientras que el **diseño modela la solución en términos físicos**, enfatizando más sobre los requerimientos de calidad (RNF). Con los nuevos métodos (RUP), la división entre el análisis y el diseño empieza a ser cada vez más difusa. Es muy difícil plantear la solución del problema sin tener en cuenta aspectos de implementación.

¿Qué es la arquitectura?

Conjunto de decisiones significativas que tomamos sobre el producto para cumplir con los requerimientos. Explica cómo se satisfacen los RF y RNF. Es el artefacto más importante para manejar los diferentes puntos de vista que pueden tener los usuarios en los diversos momentos a lo largo de la vida del proyecto y controlar el desarrollo iterativo e incremental de un sistema. La arquitectura tiene que ver con la estructura, el comportamiento, el uso, la funcionalidad, el rendimiento, la capacidad de adaptación, las restricciones, etc.

¿En qué aspectos deben diseñarse la arquitectura?

- ✓ **Arquitectura:** Responde a los requerimientos de calidad del software y toma decisiones estratégicas.
- ✓ **Datos:** Transforma los requerimientos en estructuras de datos necesarias para la persistencia del software.
- ✓ **Procesos:** Describe procedimentalmente los componentes del software.
- ✓ **Experiencia de Usuario (UX):** Relacionada con el diseño, evaluación e implementación de sistemas interactivos para uso humano.
- ✓ **Formas de Entrada/Salida:** Describe cómo se ingresa y presenta la información en el software.
- ✓ **Procedimientos Manuales:** Integra el software al sistema de negocio.

Estos aspectos aseguran que el diseño sea completo, rastreable, y que cumpla con los requisitos tanto explícitos como implícitos del cliente.

Diseño de experiencia de usuario UX: Es la disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para uso humano.

El diseño deberá:

- Implementar todos los **requisitos explícitos** del modelo de **análisis**, y deberán ajustarse a todos los requisitos **implícitos** que desea el **cliente**.
- Ser una guía **legible y comprensible** para aquellos que generen código y para aquellos que comprueban y consecuentemente dan soporte al software
- Deberá proporcionar **una imagen completa del software**, enfrentándose a los dominios de comportamiento, funcionales y de datos desde un enfoque de implementación.

Análisis	Diseño
Modelo conceptual, porque es una abstracción del sistema y permite aspectos de la implementación.	Modelo físico, porque es un plano de la implementación.
Genérico respecto al diseño (aplicable a varios diseños)	No genérico, específico para una implementación.
Menos formal.	Más formal.
Dinámico (no muy centrado en la secuencia)	Dinámico. Muy centrado en la secuencia.
Bosquejo del diseño del sistema, incluyendo su arquitectura.	Manifiesto del diseño del sistema, incluyendo su arquitectura

Un buen diseño debe considerar **Independencia de componentes** (alta cohesión y bajo acoplamiento), **Identificación y tratamiento de excepciones** y **prevención y tolerancia de defectos**.



El diseño deberá poderse rastrear hasta el modelo de análisis.



El diseño no deberá inventar nada que ya esté inventado.



El diseño deberá «minimizar la distancia intelectual» entre el software y el problema.



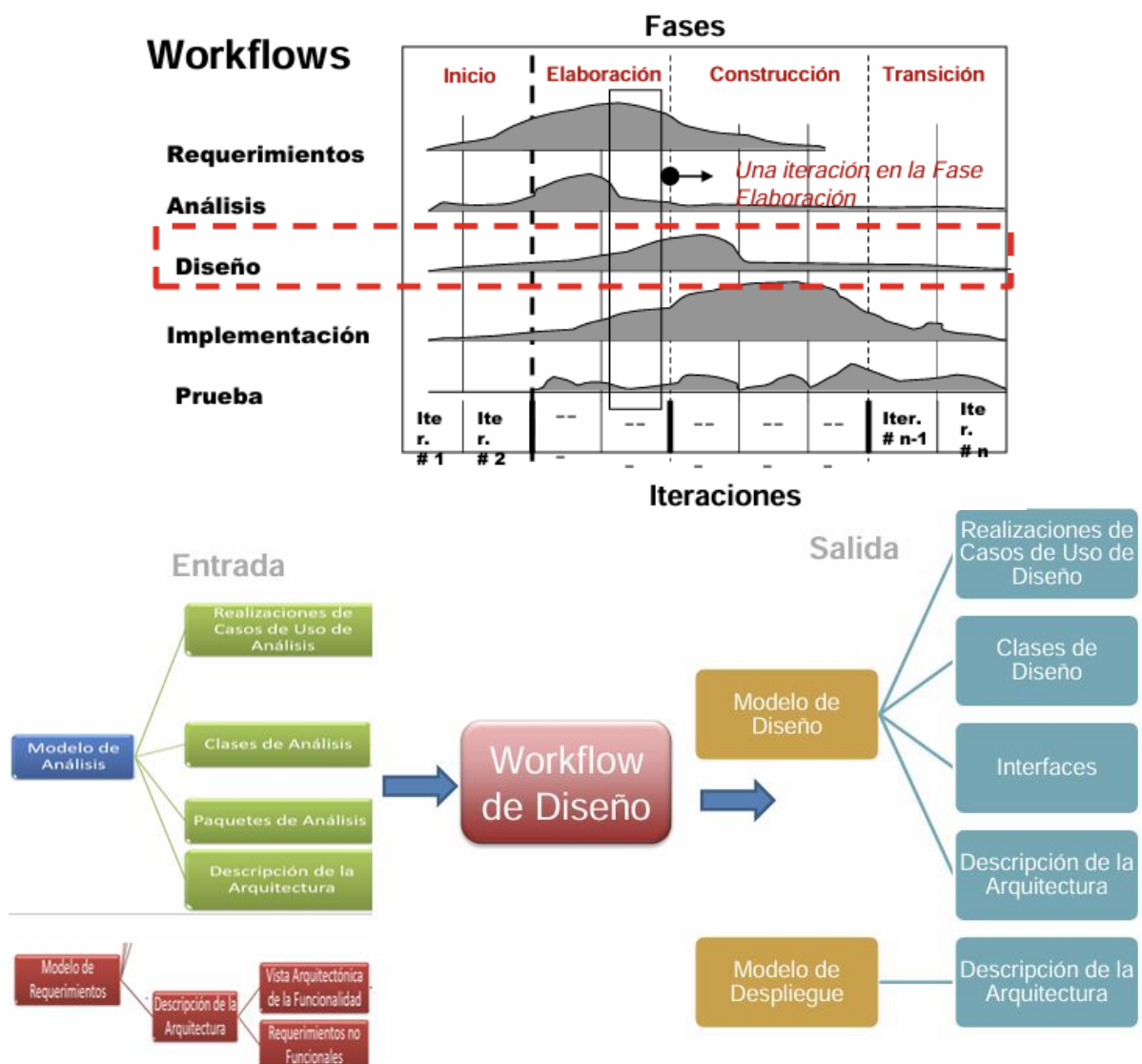
El diseño deberá presentar uniformidad e integración.

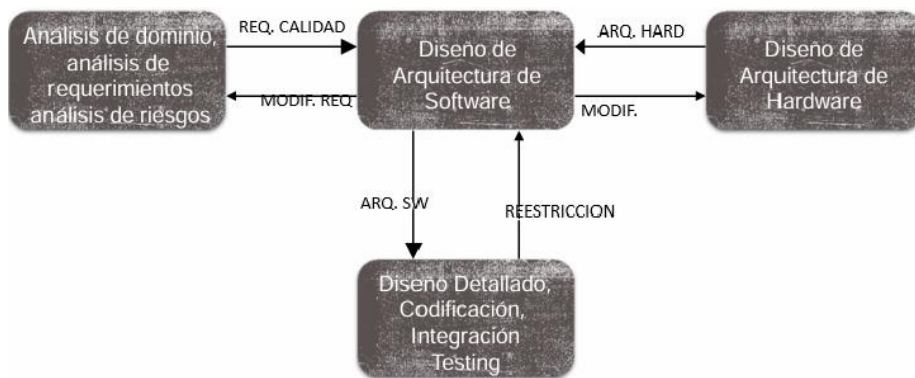


El diseño deberá estructurarse para admitir cambios.

Workflow de Diseño. Propósito.

- Adquirir una comprensión en profundidad de los aspectos relacionados con los requerimientos no funcionales y restricciones tecnológicas.
- Crear una entrada para el workflow de implementación.
- Capturar interfaces entre subsistemas.
- Descomponer los trabajos de implementación en partes manejables y permitiendo la concurrencia entre equipos de desarrollo.
- Crear una abstracción de la solución de diseño sin estar restringidos a una tecnología.





¿Qué es el diseño

Arquitectónico? Es la asignación de modelos de requerimientos esenciales a una tecnología específica → **es el plan de diseño**, es un **proceso** que toma el modelo de análisis y los RNF que salen del workflow de requerimientos y los aplica sobre el modelo esencial.

Descripción del proceso de diseño arquitectónico.

Como entrada al workflow tenemos los RNF y RF, que para poder realizar buenas decisiones estratégicas para el diseño, se los debe categorizar según su prioridad.

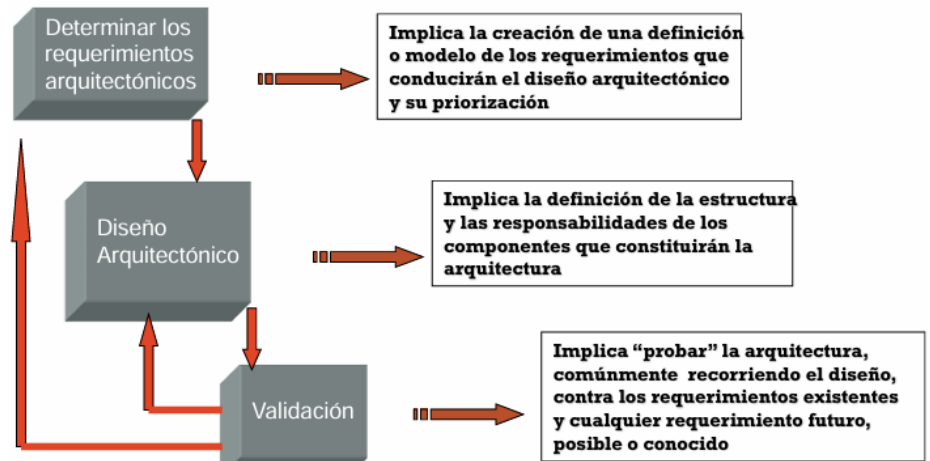
1er act: DETERMINAR LOS REQ. ARQ.

(SPA): Significativo Para la Arq.

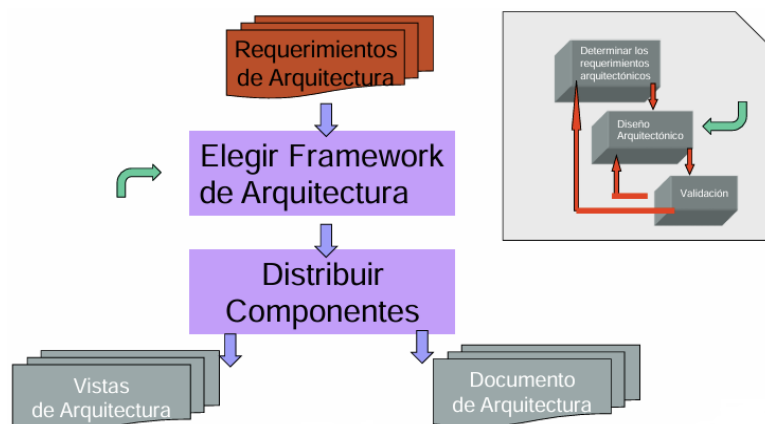
Es suficiente ubicar los requerimientos en 3 categorías.

- Alto: la aplicación debe soportar este requerimiento, conducen el diseño de la arquitectura, es por ello que se lo debe atender en la primera iteración/release(MVP)
- Medio: necesitará ser soportado en alguna etapa, pero no necesariamente en el primer reléase.
- Bajo: conforman la *wishlist*, son soluciones deseables pero no conductoras del diseño.

La priorización puede resultar engañosa o engorrosa porque los requerimientos entran en conflicto entre sí. Hay que recordar que la priorización se va a basar en los requerimientos de calidad (RNF) y no en el objetivo del sistema o el modelo de negocio de este.



2da actividad: DISEÑO ARQUITECTONICO.



asegurando estándares de calidad.

Al elegir el framework de arquitectura, un **framework suele tener patrones arquitectónicos** que colaboran a la construcción a nivel macro. Con estos patrones se pueden armar las capas o módulos de una forma más rápida,

TIPOS de PATRONES.

De bajo nivel y detallados → idiomas o para HW

De nivel medio → patrones de diseño

De alto nivel → arquitectónicos

- La mayoría de las aplicaciones responden a un pequeño número de arquitecturas probadas y bien comprendidas.
- **Utilizar soluciones conocidas minimiza los riesgos.**
- NO hay una fórmula mágica para diseñar un framework de arquitectura.
- Un pre-requisito: conocer los patrones arquitectónicos principales para abordar ciertos atributos de calidad.

3era actividad: VALIDACIÓN.

- Ayuda a **incrementar la confianza** del equipo de diseño **en que la arquitectura cumpla con su propósito.**
- Debe realizarse con las restricciones de tiempo y presupuesto del proyecto.
- El truco es ser tan riguroso y eficiente como sea posible.
- Es un desafío validar la arquitectura ya que finalmente es un diseño y no puede ejecutarse ni probarse para ver si cumple los requerimientos.
- Existen **dos técnicas** principales de validación que tienen un uso probado. Ambas ayudan a identificar fallas y debilidades, áreas de riesgo potencial para las posteriores actividades de construcción.

1) **Probar Manualmente la Arquitectura utilizando escenarios.**

2) **Construir prototipos que crea un arquetipo de la aplicación deseada.**

1. **CREACION DE ESCENARIOS** → Los escenarios están **relacionados con aspectos arquitectónicos** tales como atributos de calidad, y ellos ayudan a destacar consecuencias de las decisiones arquitectónicas que están encapsuladas en el diseño. **Son artefactos simples que implican la definición de estímulos que tendrán impacto en la arquitectura**, luego de probarlos se trabaja en base a la respuesta a esos estímulos sobre la arquitectura. Sirve para destacar las implicancias de la arquitectura en las decisiones de diseño.

Atributo de calidad	Estimulo	Respuesta
Disponibilidad	Falla la conexión de red para los mensajes de clientes	No se recibieron pedidos en una sesión de usuario por 10 minutos
Modificabilidad	Se necesita un nuevo conjunto de componentes de análisis de datos	La aplicación se reconstruye con nuevas librerías y se actualizan archivos de configuración.
Seguridad	No se recibieron pedidos en una sesión de usuario por 10 minutos	La sesión se invalida y el usuario debe registrarse nuevamente.
Escalabilidad	La carga de usuarios concurrentes se duplicó durante 3 semanas	El servidor de aplicaciones se escala a un clúster de dos máquinas para manejar el incremento de la carga de trabajo.

2. **CONSTRUCCION DE PROTOTIPOS** → dado el nivel de complejidad que puede llevar hacer un escenario en concreto, conviene utilizar prototipos que son versiones mínimas restringidas de la aplicación, creadas solamente para probar algún aspecto riesgoso, o pobremente comprendido del diseño. Utilizados para dos propósitos:
Prueba de conceptos: ¿Puede la arquitectura como fue diseñada ser construida de manera tal que satisfaga los requerimientos?

Prueba de Tecnología: ¿La tecnología elegida (middleware, aplicaciones integradas, librerías, etc.) para implementar la arquitectura se comporta como es esperado?
Estas pruebas proveen información que de otra forma no es posible evidenciar o validar.

MODELADO DE LA ARQUITECTURA.

Arquitecto arquitecta arquitecturas



ROL



PROCESO



Producto

Responsable de modelar los RF del análisis

a una arquitectura tecnológica, tratando con énfasis sobre los req. Calidad y considerando que no existe la solución perfecta en el mundo real. Siendo crucial saber el compromiso que lleva tomar decisiones estratégicas sobre la arquitectura para llevar a cabo una solución óptima para el conjunto de circunstancias existentes.

Contemplando información sobre:

- Volúmenes de datos.
- Funcionalidad demandada del negocio.
- Distribución geográfica.
- Distribución del procesamiento de datos y su almacenamiento.
- Cuales procesos se ejecutarán, en que procesadores y su nivel de comunicación entre ellos.

¿PARA QUÉ DOCUMENTAR ARQUITECTURA?

- ✓ Expresión del sistema y su evolución.
- ✓ Comunicación entre involucrados.
- ✓ Evaluación y comparación de arquitectura de una forma consistente.
- ✓ Planificación, administración y ejecución de las actividades del desarrollo del sistema.
- ✓ Expresión de las características persistentes y soporte de los principios que guían un cambio aceptable.
- ✓ Verificación de la implementación del sistema.

SISTEMAS DISTRIBUIDOS. Procesamiento distribuido.

Donde el sistema de software se ejecuta en un grupo de procesadores cooperativos integrado-conectados por una red.

- ✓ **Comparten recursos de hardware y software asociados por una red**, predominan su nivel de apertura ya que son abiertos, se diseñan sobre protocolos estándar que combinan equipamiento y software de diferentes vendedores.
- ✓ **Concurrencia:** varios procesos operando al mismo tiempo sobre diferentes computadoras de la red.

- ✓ **Tolerancia a Fallas:** la disponibilidad de varios recursos permite esta capacidad.

Escalabilidad: los sistemas pueden crecer incrementando recursos para cubrir demandas nuevas.

Tres dimensiones de la **escalabilidad**:

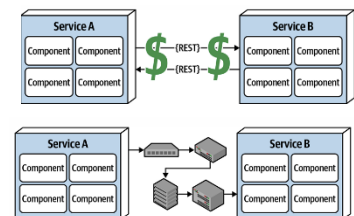
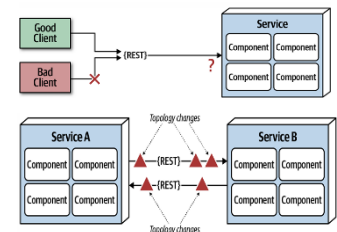
- Tamaño: debe ser posible agregar más recursos. Expandir vs Ampliar
- Distribución: dispersar los recursos sin reducir su rendimiento.
- Manejabilidad: debe poder administrarse un sistema conforme aumenta su tamaño, incluso si partes del sistema se ubican en organizaciones diferentes

Su alto nivel de complejidad hace difícil su comprensión y su prueba, del sistema, conllevando a que en caso de que ocurran defectos, se propaguen de una maquina a otra siendo mas complejo aun gestionar y administrar el mismo. La seguridad y la impredecibilidad no dejan de ser **desventajas** en los sistemas distribuidos. Los tipos de ataques que deben evitarse son:

- **Intercepción:** El atacante intercepta comunicaciones, reduciendo la confidencialidad.
- **Interrupción:** Servicios atacados con peticiones ilegítimas, impidiendo su funcionamiento.
- **Modificación:** El atacante altera datos o servicios del sistema.
- **Fabricación:** El atacante crea información falsa para obtener privilegios.

RESUMEN DE FALACIAS DE LOS SISTEMAS DISTRIBUIDOS.

- **La red está disponible:** Se asume incorrectamente que la red siempre estará operativa y sin fallos.
- **La latencia es cero:** Se cree que los datos se transfieren instantáneamente, ignorando los retrasos reales.
- **El ancho de banda es infinito:** Se supone que siempre habrá suficiente capacidad de red para cualquier cantidad de datos.
- **La red es segura:** Se asume que la red no será vulnerable a ataques o interceptaciones.
- **La topología nunca cambia:** Se cree que la estructura de la red permanecerá constante.
- **Hay un único administrador:** Se asume que solo una entidad gestionará la red.
- **El costo de transporte es cero:** Se ignoran los costos asociados con la transferencia de datos.
- **La red es homogénea:** Se cree que todos los componentes de la red son iguales y compatibles entre sí.



ARQUITECTURA MONOLÍTICA.

Aplicaciones autosuficientes que contengan toda la funcionalidad necesaria para realizar la tarea para la cual fueron diseñadas, sin contar con dependencias externas que la complementen. **La capacidad más notable que tiene es la INDEPENDENCIA.**

- **Definición:** En una arquitectura monolítica, todos los componentes y módulos se empaquetan e implementan como una sola unidad (actuando como un cuerpo singular). Esto incluye la interfaz de usuario, la lógica de negocio y el acceso a datos.

En la época trajo **ventajas** como su **fácil desarrollo**, especialmente desde cero; son **fáciles de probar** de forma unitaria o integral y su **rendimiento** suele ser rápido por el procesamiento local y no distribuido.

A su vez posee **desventajas** que no se deben dejar de lado, como la **curva de complejidad** a medida que se escala el sistema que por ende lleva a un **mantenimiento** complicado y a una **escalabilidad** totalmente integral ya que no se puede escalar partes específicas solamente.

Las capas suelen estar **altamente acopladas** y **dependen unas de otras** para funcionar correctamente. Esto puede ser visto como una *relación simbiótica*, donde cada capa necesita de las otras para operar, pero también puede generar problemas de mantenimiento y escalabilidad a largo plazo.

PATRONES ARQ. Vs. ESTILOS ARQUITECTONICOS.

- La distinción no es tajante y existen ejemplos donde es difícil distinguirlos. A veces se los ve como sinónimos.
- **Los patrones están en una escala menor que los estilos.**
- Múltiples patrones pueden aparecer en un mismo diseño.
- En contraste, un sistema usualmente tiene un único estilo arquitectónico dominante.

PATRON ARQUITECTONICO.

Expresa una organización estructural fundamental o esquema para sistemas de software.

Proporciona un conjunto de subsistemas predefinidos, especifica sus responsabilidades e incluye reglas y pautas para organizar las relaciones entre ellos.

- **Ayudan a definir características básicas y el comportamiento de una aplicación.**
- Mejora la partición y **promueve la reutilización del diseño**, dando solución a problemas recurrentes.
- Comprende un **conjunto de principios** que dan forma a una aplicación.

ESTILO ARQUITECTONICO.

Colección con nombre de decisiones de diseño arquitectónico que son aplicables en un contexto de desarrollo dado, restringen decisiones de diseño arquitectónico que son específicas de un sistema particular dentro de un contexto y obtienen cualidades beneficiosas en el sistema resultante.

- Establece un **marco de referencia a partir del cual es posible construir aplicaciones que comparten un conjunto de atributos y características mediante las cuales es posible identificarlos y clasificarlos!**
- Sirven para nombrar un diseño arquitectónico recurrente.
- **No determinan tecnología ni detalles de Implementación!**

LA REALIDAD.

PATRONES PLATÓNICOS → idealizados, se los ve en el marco teórico pero la realidad es que es imposible aplicarlo en código, es por ello que se toma como base y a medida que se lo implementa va tomando concesión que hace que el patrón termine siendo un patrón embebido.

PATRONES EMBEBIDOS → es el verdadero patrón, el que si se aplica y se ve en sistemas, violan las restricciones estrictas de los platónicos que generalmente esta violación va compensada con un gran por qué o una gran compensación que justifique.

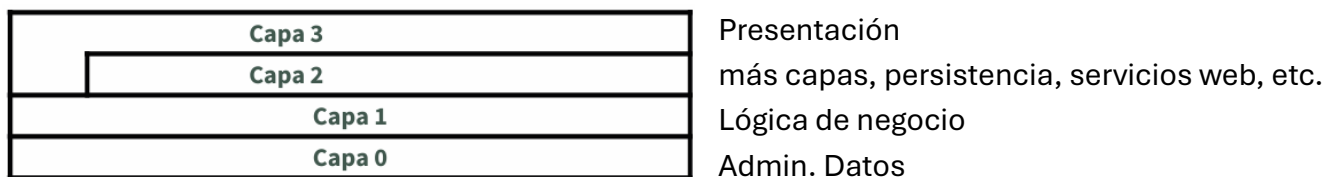
Spaghetti architecture – Big Ball of Mud.

Es **el estilo arquitectónico** que tiene un producto **cuando no se define una arquitectura**. Surge de un equipo que no tiene un plan sobre cómo se va a construir su producto.

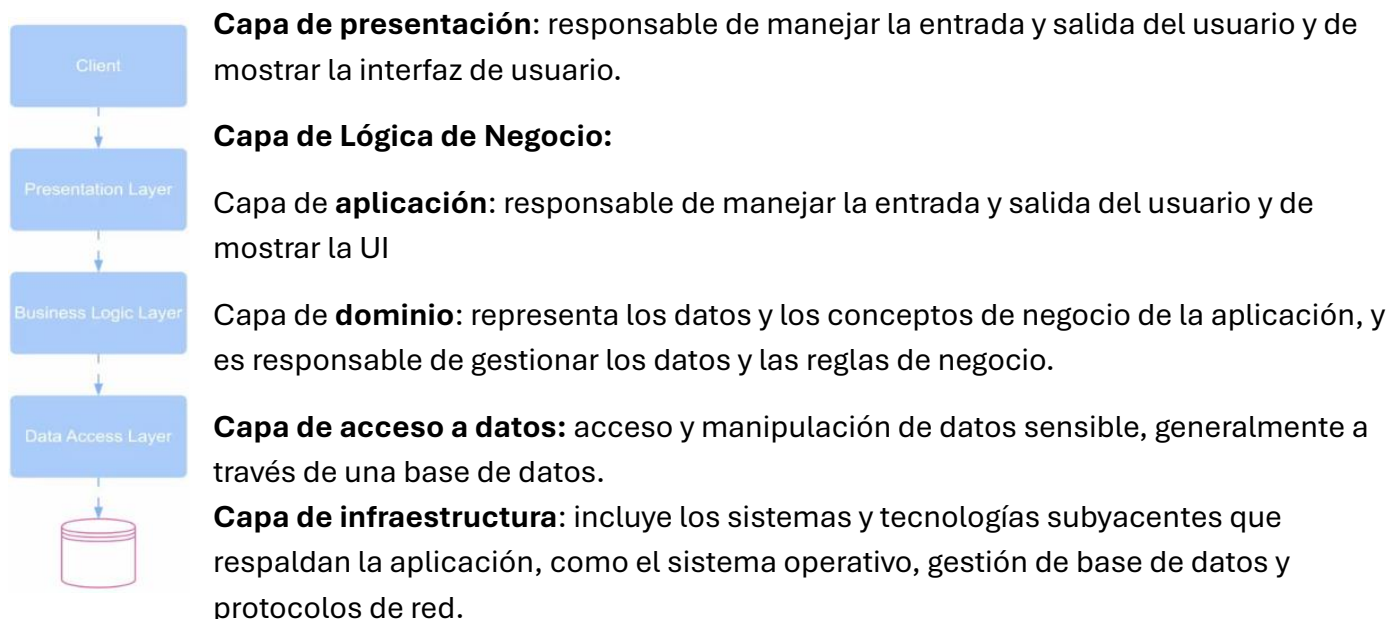
Las partes del sistema a menudo están altamente acopladas, el mantenimiento es muy difícil. Explicar cómo encaja todo a alguien nuevo es casi imposible.

PATRONES ARQUITECTONICOS -----

Arquitectura Estratificada o en capas (Layered Style).



Reconocido por ser el más común y usado en los productos de software, seccionando el mismo en **CAPAS (software → Capas | hardware → Niveles)** que son subsistemas, que tendrán componentes que serán cuidadosamente seleccionados basados en los principios de diseño POO, alta cohesión y bajo acoplamiento.



El objetivo de este patrón es definir las finalidades de cada capa y así juntar los componentes con la mayor relación posible. Hay **variantes** que **evitan la restricción de** modo que **las capas** puedan comunicarse con capas de más abajo. Otras variantes usan **capas compartidas**, donde cada capa puede usar estas capas verticales o compartidas. Este uso tensiona la definición de capas hasta el límite. No obstante, si interpretamos esas capas compartidas como una conveniencia visual para mostrar dependencia sobre un módulo compartido, esta variante tendría más sentido.

Este patrón visto desde un enfoque embebido se puede saltar capas hacia capas inferiores, lo cual provoca negar los atributos de calidad que son su beneficio. **Aun así, es beneficiosa por su característica de agrupar módulos en funcionalidad coherente.**

VENTAJAS.

- Fácil desarrollo, fácil implementación, gran conocimiento en el mundo tecnológico.
- Fácil de probar, testeo de capas por separado
- Fácil mantenimiento y detección temprana de errores

- Mejor seguridad por su separación de capas, permitiendo un aislamiento de los servidores en subredes diferentes.

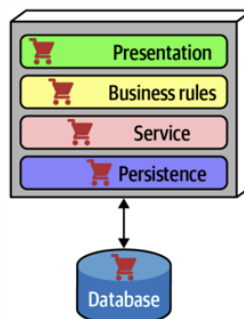
DESVENTAJAS.

- La comunicación por la red o internet es una de las tareas más lentas de un sistema, incluso, más que el mismo procesamiento de los datos, por lo que el hecho de tener que comunicarnos de capa en capa genera un **degrado significativo en el performance**.
- Al ser monolíticas, son **difíciles de escalar**, aunque es posible replicar la aplicación completa en varios nodos, lo que provoca un escalado monolítico.
- **Complejidad de despliegue:** es necesario desplegar los componentes de abajo arriba, lo que crea una dependencia en el despliegue. Un pequeño cambio puede requerir el despliegue completo de la aplicación
- **Anclado** de por vida a un solo **stack** tecnológico.
- **Tolerancia a fallos**, propensa a una falla en cascada si una capa falla.

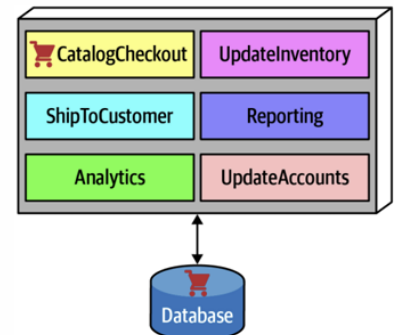
ESTILO ARQUITECTONICO: PARTICIÓN DE DOMINIO.

- **Separan los componentes de nivel superior por flujos de trabajo y/o dominios.**
- Modelado más de cerca de cómo funciona el negocio en lugar de hacia detalles de implementación.
- Se alinea más estrechamente con el monolítico modular y la arquitectura de microservicios.
- El flujo de mensajes coincide con el dominio del problema
- Fácil de migrar datos y componentes a la arquitectura distribuida

• Particionamiento Técnico

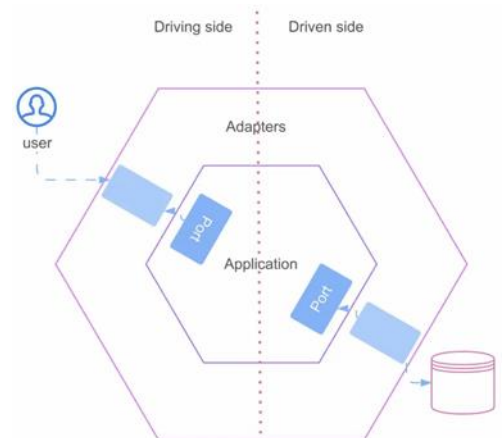


• Particionamiento de Dominio



Arquitectura hexagonal o de puertos y adaptadores.

- La lógica de negocio crea puertos que se pueden conectar a varios adaptadores de forma intercambiable.
- **Las interfaces creadas por el código de negocio se denominan puertos**, que se consideran internos tal como los define el negocio.
- **Los adaptadores representan código externo que cierra la brecha entre el puerto y el resto del código** del lado del usuario o del servidor.
- Esto crea una **capa de abstracción** que protege el núcleo de una aplicación y la aísla de herramientas y tecnologías externas.
- La arquitectura está simbolizada por un hexágono, que proporciona un amplio espacio para representar múltiples puertos y adaptadores.



PUERTOS → punto de entrada independiente de la tecnología, determina la interfaz que permitirá a los actores externos comunicarse con la Aplicación, independientemente de quién o qué implementará dicha interfaz. Los puertos también permiten que la Aplicación se comunique con sistemas o servicios externos, como bases de datos, intermediarios de mensajes, otras aplicaciones, etc.

ADAPTADORES → iniciará la interacción con la Aplicación a través de un Puerto, utilizando una tecnología específica, por ejemplo, un controlador REST representaría un adaptador que permite a un cliente comunicarse con la Aplicación. Pueden existir tantos Adaptadores para un mismo Puerto como sean necesarios sin que esto represente un riesgo para los Puertos o la propia Aplicación

Los actores conductores (**Driving side**) son los que inician la interacción y siempre están representados en el lado izquierdo. Por ejemplo, un adaptador de conducción podría ser un controlador que es el que toma la entrada (del usuario) y la pasa a la Aplicación a través de un Puerto.

Los actores conducidos (**Driven side**) son aquellos a los que la Aplicación “impulsa a comportarse”. Por ejemplo, la aplicación llama a un adaptador de base de datos para que obtenga un determinado conjunto de datos de la persistencia.

Los Adaptadores de conducción utilizarán un Puerto y un Servicio de Aplicación implementará la Interfaz definida por el Puerto, en este caso tanto la interfaz como la implementación del Puerto están dentro del Hexágono. Los adaptadores conducidos implementarán el Puerto y un Servicio de Aplicación lo utilizará; en este caso el Puerto está dentro del Hexágono, pero la implementación está en el Adaptador, por lo tanto, fuera del Hexágono.

Inversión de dependencia – Principio SOLID en Arq. Hex.

La **inversión de dependencias** es un principio de diseño de software que forma parte de los principios SOLID. Este principio sugiere que:

- **Abstracciones sobre detalles:** Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones (interfaces).
- **Interfaces y abstracciones:** Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones.

En el contexto de la arquitectura hexagonal:

- **Puertos y adaptadores:** Los puertos (interfaces) definen cómo la aplicación se comunica con sistemas externos¹. Los adaptadores implementan estos puertos.
- **Dependencia invertida:** En el lado de conducción, el adaptador depende del puerto, que es implementado por el servicio de aplicación. En el lado conducido, el servicio de aplicación depende del puerto, y el adaptador implementa la interfaz del puerto.

Esto permite que el núcleo de la aplicación esté aislado de los detalles técnicos y sea más fácil de mantener y probar.

Funcionamiento General de la arq. Hexagonal.

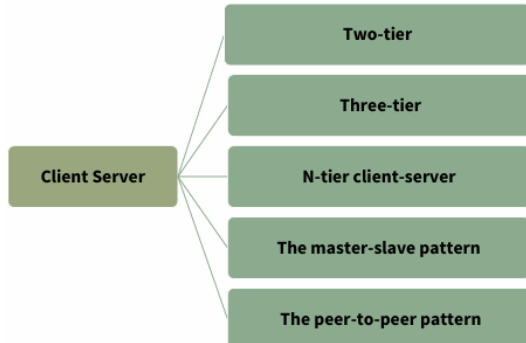
- **Núcleo de la aplicación:** contiene la lógica de negocio y los algoritmos que resuelven el problema.

- **Puertos:** define las interfaces que utiliza la aplicación para comunicarse con los sistemas externos.
- **Adaptadores:** implementan los puertos y proporcionan un puente entre la aplicación y los sistemas externos.
- **Edge:** maneja la comunicación entre el núcleo de la aplicación y los sistemas externos.
- **Sistemas externos:** interactúa con la aplicación a través de los puertos y adaptadores

ARQUITECTURA HEXAGONAL VS. LAYERED.

LAYERED	HEXAGONAL
Es criticada por su enfoque en la base de datos, lo que genera problemas como dependencias transitivas, límites de capas borrosos, aislamiento deficiente de los componentes y dificultades para probar, mantener e intercambiar componentes de infraestructura. Este enfoque se conoce como diseño basado en bases de datos, donde la aplicación se planifica en torno a una base de datos. La arquitectura en capas a menudo da como resultado mayores dependencias y una posible duplicación de la lógica de negocio	Enfatiza la lógica de negocios sobre el diseño de bases de datos. Es más lógico planificar y desarrollar primero el lado de negocio de una aplicación y considerar la persistencia de los datos más adelante. <i>Los cambios en la lógica de negocios deberían impulsar cambios en la persistencia de los datos, no al revés.</i> A medida que las aplicaciones se vuelven más complejas e incorporan componentes adicionales como API REST y servicios de terceros, la arquitectura hexagonal proporciona pautas claras sobre dónde pertenecen estos componentes.

CLIENTE/SERVIDOR: VARIANTES. DOS CAPAS.

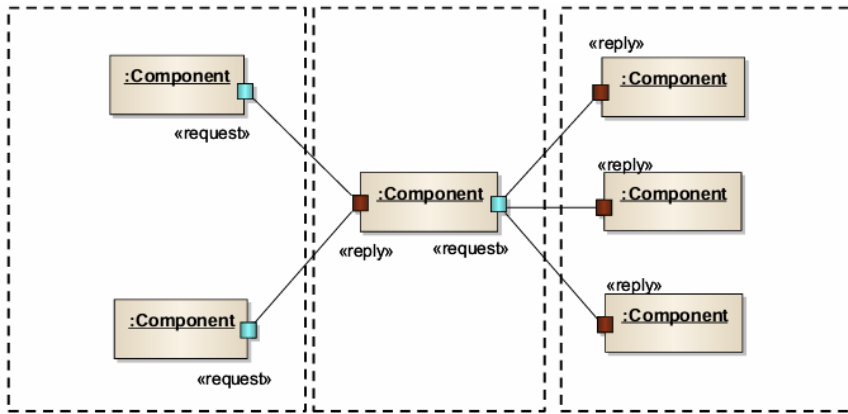


Estructura los componentes de sw en dos tipos cliente o servidor, los clientes solo hacen peticiones y los servidores solo pueden atender peticiones.

La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usan esos servicios.

Los **clientes conocen** a los **servidores** y deben saber cómo buscarlos, pero los **servidores no necesitan** conocer a los clientes.

- Clientes y Servidores son procesos lógicos.
- Los clientes inician la comunicación.
- Los servidores no conocen la identidad de los clientes hasta que se han contactado.



Cliente Ser. N-Tier.

Separación de Intereses: capas diferentes y claramente divididos para presentación, negocios y manejo de datos.

Comunicaciones sincrónicas: la comunicación entre niveles es pedido respuesta sincrónica. Los pedidos emanan en una única

dirección del nivel del cliente a los niveles web y de lógica de negocio al nivel EIS. Cada nivel espera la respuesta del otro nivel antes de proseguir.

Distribución flexible: no hay restricciones para la distribución multi- capas de la aplicación. Todas las capas pueden correr en la misma máquina o en su propia máquina. En aplicaciones WEB, el cliente usualmente corre en un browser de una PC de escritorio de un usuario, comunicándose remotamente con Internet con componentes a nivel web.

VENTAJAS:

- Información completamente centralizada y al día.
- Seguridad ampliada con firewall o subredes que dificultan el acceso a BDA.
- Fácil instalación para el cliente, por lo general es una aplicación simple sin dependencias.
- Esta arquitectura separa la lógica de negocio de los clientes.
- Alta portabilidad ya que al ser dos aplicaciones se pueden desarrollar con diferentes tecnologías para múltiples plataformas.

DESVENTAJAS:

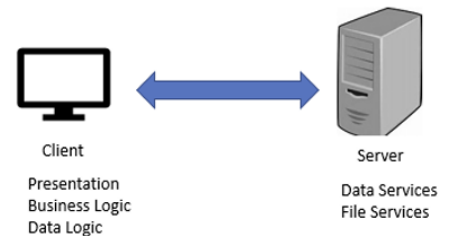
- Cliente pesado, complicada gestión de varios clientes al subir una actualización.
- Sin estrategia de escalamiento en caso de que varios clientes realicen peticiones en simultaneo, la cantidad de solicitudes a procesar al mismo tiempo puede causar fallos en rendimiento o estabilidad.
- Si el servidor cae, los clientes quedan inoperables.
- Los protocolos de comunicación entre cliente y servidor suelen ser de bajo nivel (Sockets, HTTP, RPC).
- Complejidad al querer realizar una trazabilidad de errores en la depuración de estos, por tener clientes distribuidos en varias máquinas.

Un estilo fundamental en la arquitectura separa la funcionalidad técnica entre frontend y backend, denominada arquitectura de dos niveles o cliente/servidor.

Formas destacadas de implementar el patrón C/S:

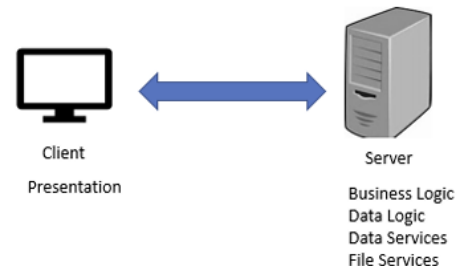
El patrón cliente-servidor de acceso a datos remoto

- En este enfoque, el cliente (generalmente una aplicación o interfaz de usuario) se comunica con un servidor remoto para acceder a datos almacenados en una base de datos o sistema de archivos.
- El cliente envía solicitudes al servidor para recuperar, actualizar o eliminar datos.
- El servidor procesa estas solicitudes y devuelve los resultados al cliente.
- Ejemplos comunes incluyen aplicaciones web que se conectan a una base de datos en un servidor remoto.



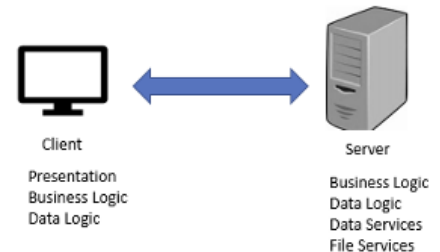
El patrón cliente-servidor de presentación remota

- En este caso, el servidor maneja la lógica de presentación (como la generación de páginas HTML) y envía los resultados al cliente.
- El cliente puede ser un navegador web o una aplicación de escritorio.
- El servidor procesa las solicitudes del cliente y genera la interfaz de usuario (por ejemplo, páginas web) para su visualización.
- Este enfoque es común en aplicaciones web donde el servidor genera contenido dinámico para el cliente.



El patrón cliente-servidor de datos de lógica dividida

- Aquí, la lógica de negocio se divide entre el cliente y el servidor.
- El cliente maneja la interfaz de usuario y algunas operaciones lógicas.
- El servidor se encarga de procesos más complejos, como cálculos, validaciones o acceso a bases de datos.
- Este enfoque permite una distribución más equilibrada de la carga de trabajo entre cliente y servidor.



PATRON MAESTRO ESCLAVO o master-slave.



Se usan comúnmente en sistemas de tiempo real donde es importante cumplir con los tiempos de procesamiento.

El proceso “maestro” es por lo general el responsable de la computación, coordinación y comunicación y controla los procesos “esclavos”.

Los procesos “esclavos” se dedican a acciones específicas, como adquisición de datos de sensores. Se usa cuando es posible predecir el procesamiento distribuido que se requiere y cuando se puede asignar fácilmente el procesamiento a los procesadores esclavos.

Arquitectura Peer-to-Peer.

La **arquitectura peer-to-peer (P2P)** es un estilo en el que los nodos se comunican entre sí como iguales, sin relaciones jerárquicas. Aquí tienes un resumen de sus características principales:



- **Comunicación entre pares:** Cada nodo puede actuar como cliente y servidor, solicitando y proporcionando servicios a otros nodos.
- **Disponibilidad** desde cualquier nodo a un archivo, con una **resistencia** a fallos que afecten el sistema por completo. Siendo también **altamente escalable** y extensible con un uso **eficiente** de **recursos** de hardware, capaces de aprovechar la capacidad de procesamiento de computadores locales sin uso.
- **Descentralización:** No hay un servidor central, lo que permite una mayor autonomía y privacidad.

¿Cuándo usar P2P? Sistemas de cómputo intensivo y aquellos que requieren intercambio de información entre computadoras sin necesidad de almacenamiento centralizado.

Ventajas:

- **Alta escalabilidad:** Pueden escalar fácilmente, ya que a mayor número de nodos, más recursos hay disponibles y la carga de trabajo se divide entre todos los participantes.
- **Tolerancia a fallas:** Tener los recursos distribuidos por varios nodos permite que los recursos estén disponibles sin importar si algunos de los nodos se caen.
- **Descentralización:** Los sistemas P2P puros tienen una autonomía completa, por lo que pueden funcionar sin un servidor central que organice toda la red.
- **Privacidad:** Debido al sistema de búsqueda por inundación o tablas hash, es posible tener un alto nivel de privacidad, pues no es tan fácil saber qué nodo fue el que realizó la petición inicial.
- **Equilibrio de carga:** Todos los nodos de la red tienen el mismo rol y responsabilidades, por lo que toda la carga de trabajo se equilibra entre todos los nodos de la red.

Desventajas:

- **Alta complejidad:** Tiene una complejidad muy alta para desarrollarse, ya que se deben crear aplicaciones que funcionen como cliente y servidor, al mismo tiempo que se debe garantizar que las búsquedas de los recursos sean eficientes.
- **Control:** A menos que se tenga un servidor central para administrar los recursos que se buscan y los usuarios que se conectan a la red, es muy fácil perder el control sobre el contenido que se puede compartir en la red.

- **Seguridad:** Esta es una de las características menos implementadas en este tipo de arquitecturas, para evitar la interceptación de las comunicaciones, nodos maliciosos, contenido falso o adulterado, o la propagación de virus o programas maliciosos.
- **Tráfico:** En redes no optimizadas como las no estructuradas, se puede saturar la red con una gran cantidad de tráfico para propagar las peticiones a los nodos adyacentes.

ESTILOS PARA EL DISEÑO DE APLICACIONES WEB.

The model-view-controller MVC → Separa la presentación y la interacción de los datos del sistema, permitiendo múltiples formas de ver e interactuar con los datos.

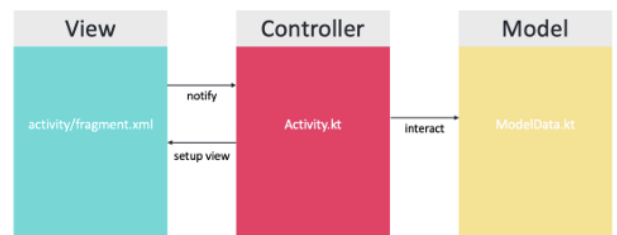
The model-view-presenter MVP → Similar a MVC, pero con un enfoque en la separación de la lógica de presentación y la lógica de negocio.

The model-view-model MVVM → Utiliza bindings para sincronizar automáticamente la vista y el modelo, facilitando la gestión de la interfaz de usuario.

The front controller → Centraliza las solicitudes de la aplicación en un único controlador, mejorando la gestión y el enrutamiento de las solicitudes.

The model-view-controller MVC

Basado en 3 tipos de objetos.



- **Modelo:** Es responsable de la **lógica empresarial** de la aplicación. Gestiona el estado de la aplicación, incluyendo la lectura y escritura de datos, el estado persistente y tareas relacionadas con la gestión de datos, como la red y la validación de datos.
- **Vista:** Presenta los **datos al usuario** y maneja la interacción del usuario. Se asegura de que su apariencia refleje el estado del modelo.
- **Controlador:** Une la capa de vista y la capa de modelo. **Define cómo la interfaz reacciona** a la entrada del usuario y maneja la lógica que no pertenece ni al modelo ni a la vista.

Ventajas de la Arquitectura MVC:

- **Separación de responsabilidades:** Permite que los datos cambien de manera independiente de su representación y viceversa.
- **Múltiples vistas:** Soporte a distintas representaciones de los mismos datos, y los cambios en una representación se muestran en todas ellas.

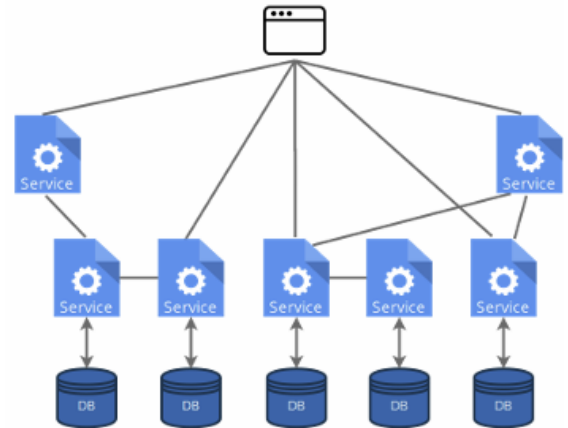
Desventajas de la Arquitectura MVC:

- **Complejidad adicional:** Puede implicar código adicional y complejidad de código cuando el modelo de datos y las interacciones son simples.
- **Curva de aprendizaje:** Requiere una comprensión clara de la separación de responsabilidades entre el modelo, la vista y el controlador.

Este patrón ayuda a **separar la presentación e interacción** de los datos del sistema, facilitando la **modularidad, flexibilidad y mantenibilidad** de la aplicación.

Service Oriented Architecture SOA.

SOA divide programas monolíticos en servicios independientes, cada uno con una función específica. Estos servicios se comunican a través de una capa API definida formalmente.



- **Interoperabilidad:** Los servicios están diseñados para ser integrados con otros, utilizando estándares abiertos, lo que permite la comunicación entre aplicaciones heterogéneas.
- **Flexibilidad y Reutilización:** La flexibilidad de SOA permite construir nuevos servicios utilizando la infraestructura existente, promoviendo la reutilización y la agilidad en el desarrollo.
- **Desventajas:** Aunque SOA reduce el acoplamiento y mejora la escalabilidad, puede presentar desafíos en términos de rendimiento y gobernanza, además de aumentar los puntos de falla potenciales.

Composición de los servicios.

- **Construcción de Nuevos Servicios:** La flexibilidad de SOA permite construir nuevos servicios utilizando la infraestructura existente. Esto facilita la creación de aplicaciones más completas y personalizadas.
- **Enfoque en Flexibilidad:** SOA prioriza la flexibilidad sobre la optimización. Se centra en crear servicios reutilizables que puedan adaptarse a diferentes contextos y necesidades.
- **Latencia en Consumo de Servicios:** Aunque SOA ofrece flexibilidad, no es especialmente rápida debido a la latencia inherente al consumo de servicios distribuidos.
- **Niveles de Composición:** La composición de servicios puede variar en granularidad. Por ejemplo, los niveles más básicos pueden ser servicios CRUD (Altas, Bajas, Cambios, Borrado), mientras que niveles más avanzados pueden involucrar operaciones completas, como reservar un vuelo con hospedaje y alquilar un automóvil.

SOA con Service Bus (ESB)

El service bus crea una abstracción de los servicios expuestos para evitar un acoplamiento y a su vez evita las conexiones punto a punto entre servicios, siendo el intermediario. Esto da la posibilidad de agregar lógica adicional como auditoria, seguridad, tiempos de respuesta, etc.

SOA DESVENTAJAS.

1. **Volumetría:** Cuando se necesita procesar grandes cantidades de información, SOA puede no ser la mejor opción debido a la latencia en el consumo de servicios distribuidos. En tales casos, considerar otras arquitecturas más adecuadas para manejar cargas pesadas puede ser recomendable.

2. **Gobierno:** Es fundamental establecer un gobierno sólido en una arquitectura SOA. Esto implica definir políticas y reglas para el uso y la creación de nuevos servicios. Un gobierno bien estructurado ayuda a mantener la coherencia, la calidad y la seguridad de los servicios.
3. **Más Puntos de Falla:** Al componer servicios, se aumenta la complejidad y, por lo tanto, la posibilidad de fallos. Para mitigar esto, es necesario implementar estrategias de manejo de fallos, como redundancia, monitoreo constante y recuperación ante errores.

SOA VENTAJAS.

1. **Reduce el Acoplamiento:** En SOA, los servicios se comunican entre sí a través de interfaces estándar. Esto reduce el acoplamiento entre componentes, lo que significa que un servicio puede estar en cualquier parte y aún así interactuar con otros servicios de manera eficiente.
2. **Testabilidad:** Los servicios SOA son fáciles de probar de forma individual. Puedes evaluar cada servicio por separado para asegurarte de que funcione correctamente antes de integrarlo en una aplicación más grande.
3. **Reutilización:** Los servicios SOA están diseñados para realizar tareas específicas. Esto facilita la reutilización de servicios existentes en diferentes contextos y aplicaciones.
4. **Agilidad:** SOA permite crear nuevos servicios a partir de otros ya existentes. Esto agiliza el desarrollo y la adaptación de aplicaciones según las necesidades cambiantes.
5. **Escalabilidad:** Puedes agregar múltiples instancias de un servicio y balancear la carga utilizando un ESB (Enterprise Service Bus). Esto garantiza que la arquitectura sea escalable y pueda manejar aumentos en la demanda.
6. **Interoperabilidad:** SOA utiliza estándares abiertos como SOAP y REST, lo que permite que aplicaciones heterogéneas se comuniquen de manera homogénea. Esto es esencial para la integración de sistemas en entornos empresariales complejos.

Event Driven Architecture.

La Arquitectura Orientada a Eventos (EDA)

se basa en la publicación, captura, procesamiento y almacenamiento de eventos.

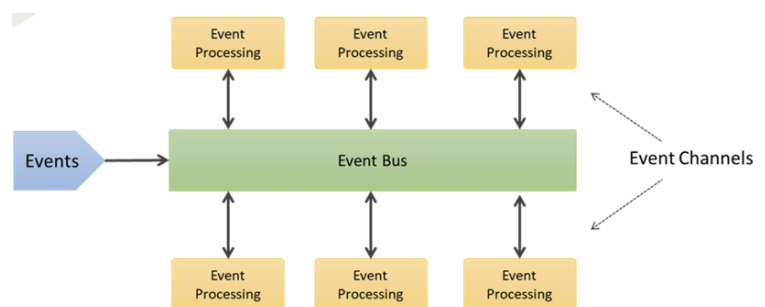
Un evento lo podemos definir como un **cambio significativo en el estado de la aplicación**, ya sea por un dato que cambio o

alguna acción concreta en el sistema que merece la pena ser observada para tomar acciones en consecuencia.

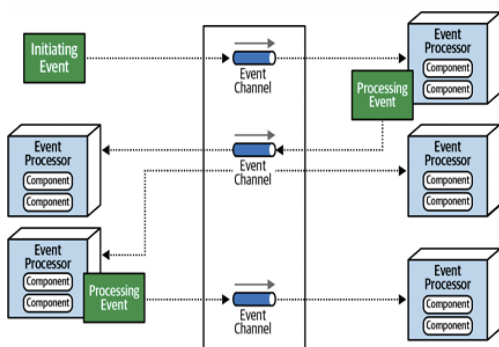
En una arquitectura EDA, las aplicaciones generan eventos y otros componentes reaccionan ante ellos. Estos componentes pueden procesar los eventos y, a su vez, generar nuevos eventos para continuar con el flujo de trabajo.

- **Características Clave:**

- **Asincronía y Distribución:** EDA se basa en la comunicación asíncrona entre componentes. Esto permite que las aplicaciones sean altamente escalables y flexibles.



- **Escalabilidad:** Al separar los componentes y permitir que reaccionen a eventos específicos, EDA facilita la escalabilidad de las aplicaciones.
- **Independencia de Fallos:** Si un componente falla, otros pueden seguir funcionando sin interrupciones.
- **Enfoque en Eventos:** Los eventos son el corazón de EDA. Cada cambio relevante en la aplicación se convierte en un evento que desencadena acciones.



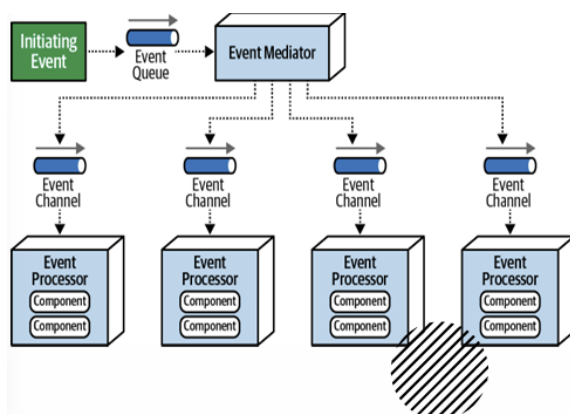
Event Driven Architecture (Con topología Broker).

Topología de Broker:

En EDA, la topología de **Broker** introduce un intermediario (el “broker”) entre los editores de eventos y los suscriptores. El broker recibe eventos de los editores y los distribuye a los suscriptores relevantes según sus intereses y suscripciones.

Ventajas de usar la topología de Broker:

- **Escalabilidad:** Al separar los editores y suscriptores, el patrón de Broker permite escalar fácilmente el sistema a medida que crece.
- **Reducción de Tráfico de Red:** Los suscriptores solo reciben los eventos en los que están interesados, lo que mejora el rendimiento y reduce la carga de la red.
- **Almacenamiento Temporal:** El broker puede almacenar eventos hasta que los suscriptores estén listos para recibirlos, garantizando una entrega confiable.



Event Driven Architecture (Con topología Mediator)

Control Centralizado: En esta topología, existe un control central que gestiona el flujo de eventos. Los mediadores de eventos determinan qué procesadores se llaman y en qué orden cuando un evento ingresa al sistema.

Múltiples Mediadores: Puede haber varios mediadores de eventos trabajando en paralelo o de forma jerárquica, lo

que permite una gestión más organizada y eficiente de los eventos.

Ventajas: Esta estructura facilita el cambio, la administración y el monitoreo del proceso, ya que encapsula la lógica del proceso en un solo lugar.

Aplicaciones: Es comúnmente utilizada para implementar procesos de negocio complejos que requieren la publicación de requerimientos a varios componentes servidores diferentes.

EDA VENTAJAS.

- **Escalabilidad:** Permite que cada consumidor de eventos escale de forma independiente, reduciendo al máximo el acoplamiento entre componentes.
- **Despliegue:** Gracias al bajo acoplamiento, los componentes pueden desplegarse sin preocuparse por dependencias, ya que solo se suscriben para recibir eventos y reaccionar a ellos.

- **Performance:** Aunque EDA puede agregar retrasos debido a su naturaleza asíncrona, el procesamiento en paralelo ayuda a superar esta desventaja.
- **Flexibilidad:** Permite responder rápidamente a cambios, ya que cada componente tiene una sola responsabilidad y está desacoplado de los demás, facilitando la adaptación a nuevos requerimientos sin afectar al resto del sistema.

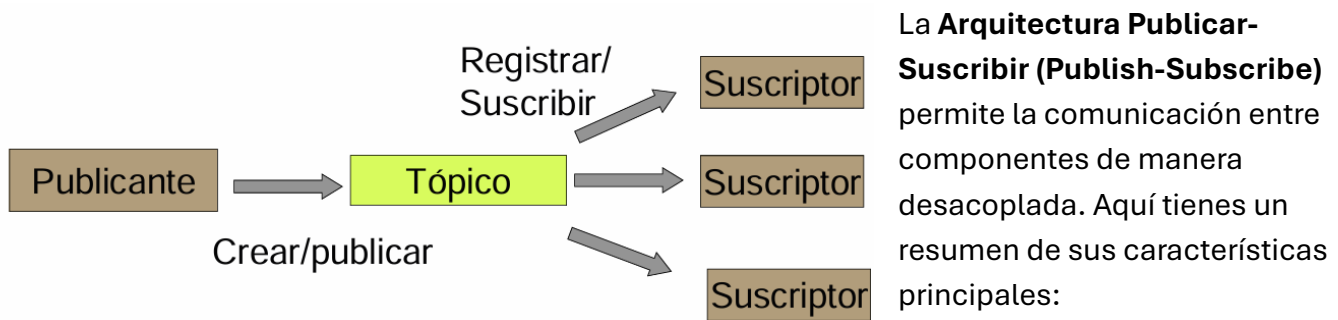
EDA DESVENTAJAS.

- **Testabilidad:** La arquitectura distribuida y asíncrona añade complejidad a las pruebas. No se puede simplemente generar un evento y esperar un resultado para validarlo. En su lugar, se necesitan **pruebas más sofisticadas** que creen eventos y esperen la finalización del procesamiento de todos los eventos generados como consecuencia del evento inicial para validar el resultado final.
- **Desarrollo:** Codificar **soluciones asíncronas es complicado**. Además, es necesario crear manejadores de errores avanzados que permitan la recuperación en una arquitectura asíncrona. En este tipo de arquitectura, la falla en un procesador no implica la falla en el resto, lo que puede llevar a inconsistencias entre los sistemas.

Variantes de Event Driven Architecture.

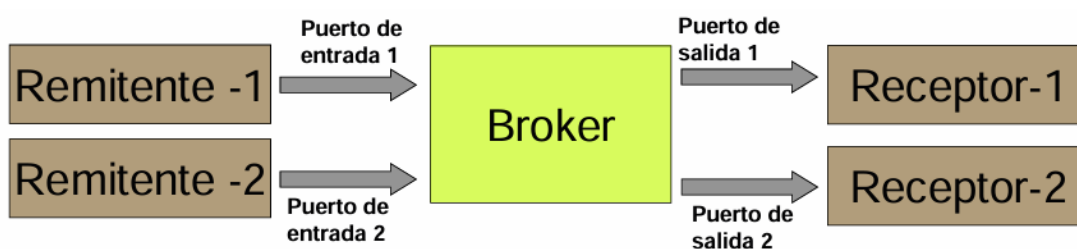
- **Publish & Suscribe**
 - En este modelo, los **emisores** (publishers) **envían mensajes a un canal sin saber quién los recibirá**. Los **receptores** (subscribers) **se suscriben a los canales de interés** y reciben los mensajes correspondientes. Es útil para sistemas donde los emisores y receptores están desacoplados.
- **Broker**
 - En esta variante, **un intermediario (broker) gestiona la comunicación entre emisores y receptores**. Los emisores envían mensajes al broker, que luego los distribuye a los receptores adecuados. Esto permite una mayor flexibilidad y escalabilidad en la comunicación.
- **Messaging**
 - Este modelo **se basa en el intercambio de mensajes entre componentes del sistema**. Los mensajes pueden ser enviados **de manera síncrona o asíncrona**, y se utilizan para coordinar acciones y compartir información entre los componentes.
- **Process Coordinator**
 - En esta variante, un coordinador de procesos gestiona la ejecución de tareas y la comunicación entre componentes. El coordinador recibe eventos, decide qué acciones tomar y coordina la ejecución de procesos en respuesta a esos eventos.

Publish & Suscribe.



- **Independencia:** Los componentes que publican eventos (publicantes) y los que se suscriben a ellos (suscriptores) son independientes entre sí.
- **Desconocimiento Mutuo:** Los publicantes no necesitan saber quién se suscribe a sus eventos, y los suscriptores no necesitan saber quién publica los eventos.
- **Flexibilidad Dinámica:** Tanto los publicantes como los suscriptores pueden aparecer y desaparecer dinámicamente, lo que permite una configuración flexible. Así como también pueden suscribirse y des-suscribirse a los tópicos de manera dinámica.
- **Mensajería Muchos a Muchos:** Los mensajes publicados se envían a todos los suscriptores registrados en un tópico. Varios publicantes pueden publicar en el mismo tópico y varios suscriptores pueden escuchar en el mismo tópico.
- **Calidad de Servicio Configurable:** Puede ser punto-a-punto o multicast/broadcast, permitiendo mensajes seguros o no seguros.
- **Consideraciones:** Ideal para aplicaciones que requieren mensajes asincrónicos uno-a-muchos, muchos-a-uno, o muchos-a-muchos. Sin embargo, puede afectar la performance debido a la capa de indirección entre productores y consumidores.

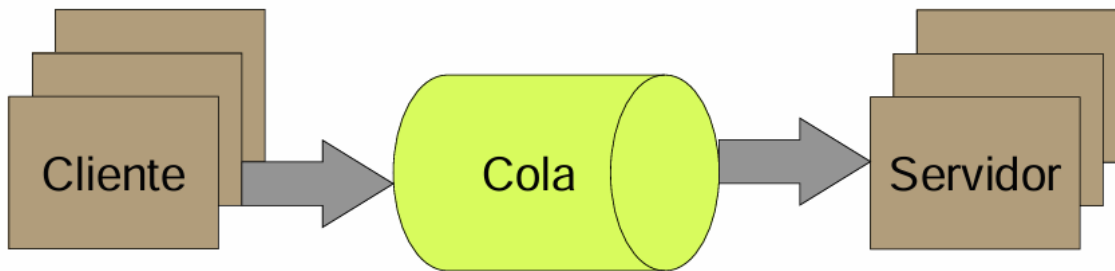
Agente (Broker).



Arquitectura Hub-and-Spoke → El agente actúa como un concentrador de mensajes, conectando remitentes y receptores a través de puertos específicos para cada formato de mensaje.

- **Ruteo de Mensajes:** El agente contiene la lógica necesaria para entregar mensajes desde un puerto de entrada a un puerto de salida.
- **Transformación de Mensajes:** El agente transforma el tipo de mensaje recibido en el puerto de entrada al tipo requerido por el puerto de salida.
- **Desacoplamiento:** Los agentes permiten que remitentes y receptores usen sus formatos nativos, centralizando la transformación lógica en el agente para facilitar su comprensión y modificación.

Arquitectura Comunicando (Messaging).



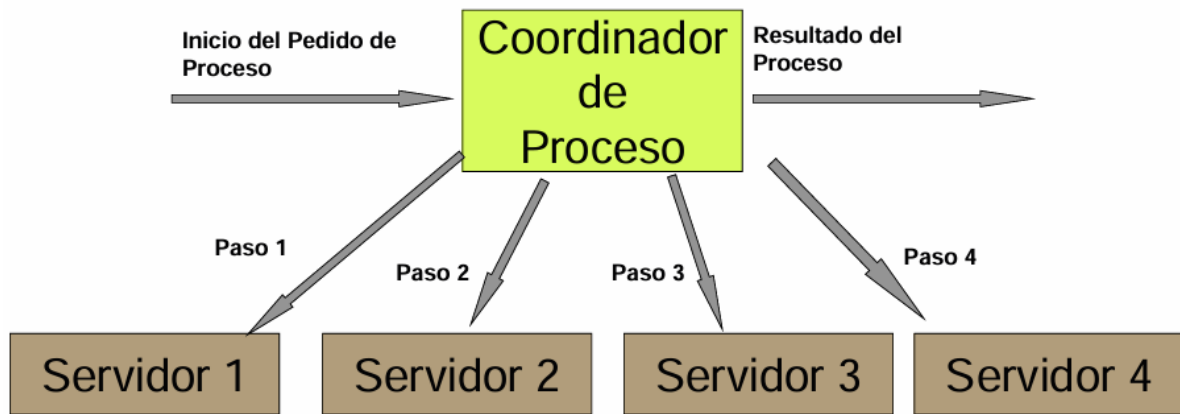
Desacoplamiento: Los emisores envían mensajes a una cola intermedia, y los receptores los reciben cuando están disponibles. Esto elimina la necesidad de una conexión directa entre emisores y receptores, permitiendo que cada uno opere de manera independiente.

- ✓ **Alta Modificabilidad:** Gracias al bajo acoplamiento, es fácil modificar emisores y receptores de manera independiente. Esto facilita la actualización y mantenimiento del sistema sin afectar a otros componentes.
- ✓ **Tolerancia a Fallos:** Los mensajes se almacenan en la cola hasta que el receptor esté disponible, asegurando la entrega eventual incluso si hay fallos en la red o el receptor no está disponible en el momento.
- ✓ **Dependencia en Formatos:** Los cambios en los formatos de mensajes pueden requerir ajustes en emisores y receptores. Sin embargo, el uso de formatos auto-descriptivos puede mitigar este problema, facilitando la interoperabilidad.
- ✓ **Comunicaciones Asincrónicas:** Los clientes envían pedidos a una cola y continúan con sus operaciones sin esperar una respuesta inmediata, lo que mejora la eficiencia y la capacidad de respuesta del sistema.
- ✓ **Calidad de Servicio (QoS) Configurable:** La cola de mensajes puede configurarse para diferentes niveles de velocidad y confiabilidad, adaptándose a las necesidades específicas de la aplicación.
- ✓ **Bajo Acoplamiento:** No hay un vínculo directo entre clientes y servidores, lo que permite una mayor flexibilidad y resistencia del sistema.

Consideraciones:

- Ideal para aplicaciones que pueden dividir el procesamiento en pasos discretos.
- Proporciona una solución resistente para aplicaciones con conectividad transitoria.
- Puede implementarse para pedido-respuesta sincrónico utilizando un par de colas.

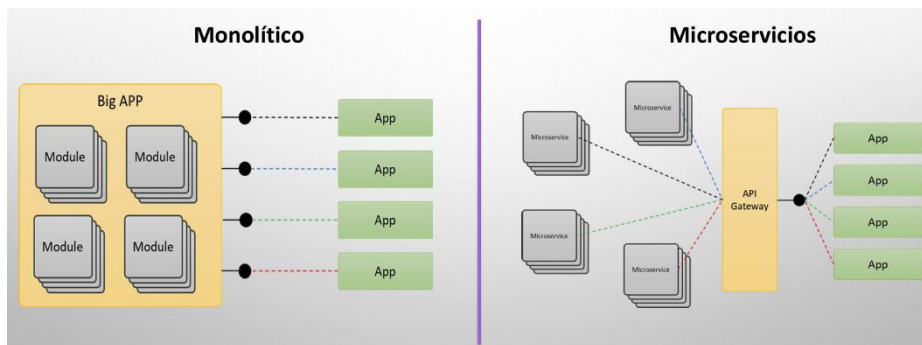
Coordinador de proceso (Process Coordinator).



Encapsulamiento de Proceso: El coordinador de proceso encapsula la secuencia de pasos necesarios para completar un proceso de negocio, facilitando su comprensión y modificación.

- **Bajo Acoplamiento:** Los componentes del servidor no conocen su rol en el proceso general, lo que permite que el coordinador los llame según sea necesario.
- **Comunicaciones Flexibles:** Las comunicaciones entre el coordinador y los servidores pueden ser sincrónicas o asincrónicas, adaptándose a diferentes necesidades.
- **Consideraciones:** Este patrón es útil para implementar procesos de negocio complejos, permitiendo cambios, administración y monitoreo más sencillos.

Arquitectura de Microservicios.



En este enfoque, una aplicación se divide en **pequeños servicios independientes**, cada uno ejecutándose en su propio proceso. Estos servicios se comunican entre sí mediante **mecanismos ligeros** (como HTTP o mensajería).

Se basa en la **modularidad**, la **autonomía** y la **escalabilidad**, lo que permite a las organizaciones desarrollar sistemas más flexibles y adaptables.

PRINCIPIOS CLAVE.

- **Componentes Independientes:** Cada módulo funciona como un servicio autónomo, interactuando con otros cuando es necesario. Cada servicio **se despliega de manera independiente**. Cada microservicio es un **módulo independiente** que realiza una **tarea específica** y se comunica con otros servicios a través de APIs.
- **Relación con SOA:** Basado en la arquitectura orientada a servicios (**SOA**), con servicios conectados a través de interfaces definidas.

- **Funciones Únicas y Bases de Datos Separadas:** Cada servicio en una aplicación de microservicio controla **una función única** y tiene **su propia base de datos**. Aunque esto puede generar datos redundantes, garantiza un **acoplamiento bajo**.
- **Diversidad Tecnológica:** Los **servicios pueden estar escritos en diferentes lenguajes** de programación y utilizar distintas tecnologías
- **Capacidades de negocio:** Los servicios se construyen alrededor de funciones específicas del negocio.
- **Enfoque de Producto, No de Proyecto:** Los equipos son responsables del software en producción desde su desarrollo hasta su operación.
- **Gobernanza Descentralizada:** Fomenta el uso compartido de código y bibliotecas para resolver problemas comunes.
- **Gestión de Datos Descentralizada:** Implementa la persistencia políglota o el uso de bases de datos variadas.
- **Automatización de Infraestructura:** La implementación y gestión de cada microservicio se automatizan en diferentes entornos. El despliegue se realiza de forma automatizada.
- **Diseño para el Fracaso:** Los microservicios están diseñados para manejar y recuperarse de fallos efectivamente.

Compartiendo código.

Los microservicios evitan la reescritura de código al compartirlo, reducen la latencia al mantener funciones en la misma aplicación, y mejoran la colaboración con una pila tecnológica uniforme. Además, los equipos comparten experiencia y los costos de infraestructura son menores, mientras que la integración se maneja durante el desarrollo, evitando bloqueos.

Los microservicios presentan desafíos como el riesgo de que cambios en el código afecten a todo el sistema y la limitación de estándares de codificación. El compartir código puede prolongar los tiempos de compilación y dificultar la refactorización. Además, la infraestructura dependiente impide la escalabilidad independiente de las partes, la uniformidad tecnológica limita las opciones, y los cuellos de botella pueden afectar al sistema en su conjunto.

Ventajas.

- **Alta Escalabilidad:** Los servicios pueden escalarse de forma independiente.
- **Agilidad y Facilidad de Despliegue:** Permite actualizaciones rápidas y despliegues continuos.
- **Testeabilidad y Reusabilidad:** Cada servicio se puede probar y reutilizar por separado.
- **Interoperabilidad:** Uso de protocolos como REST para asegurar la comunicación entre servicios.
- **Reusabilidad:** el código de implementación probado y útil se encuentra en bibliotecas, alentando a los desarrolladores a implementar microservicios de formas similares.

Desventajas.

- **Performance:** La comunicación entre servicios puede introducir latencia.
- **Múltiples Puntos de Falla:** Cada interacción entre servicios puede fallar.
- **Trazabilidad:** Dificultad para rastrear la ejecución completa de un proceso, porque cada microservicio registra su traza de forma separada, por lo que se tienen que recopilar y unir para formar una completa.

Aspectos Arquitectónicos	Microservicios	Monolítica
Implementación	Difícil	Fácil
Despliegue	Fácil	Difícil
Escalabilidad	Escalable fácilmente	Difícil
Administración	Descentralizada	Centralizada
Orientada a Negocios	Si	Levemente

- **Madurez del Equipo:** Requiere equipos bien capacitados y organizados.

DIFERENCIAS ENTRE MONOLÍTICAS Y MICROSERVICIOS.

Aspecto arquitectónico	Monolíticas	Microservicios
Implementación.	La implementación es centralizada y más fácil de desarrollar e implementar desde 0, pero si la aplicación crece en tamaño se vuelve compleja la implementación de cambios .	Enfoque descentralizado , la implementación de cada módulo por separado es fácil, pero como conjunto es compleja .
Despliegue.	Las aplicaciones monolíticas requieren desplegar la aplicación por completo en cada cambio, y se complejizan a medida que crecen, por lo que no se adaptan al “despliegue continuo”.	El despliegue de cada módulo se puede hacer independientemente de los otros sin compilar toda la aplicación. Facilita CI/CD.
Escalabilidad.	Fáciles de escalar en un principio, a medida que crecen se vuelve difícil (escalamiento completo de la aplicación).	Escalamiento independiente de cada microservicio , sin afectar el resto de aplicación. Mejora las prácticas ágiles e incorporación de servicios.
Mantenimiento y Administración.	Difícil de mantener y administrar a medida que crece, comunicación de los problemas fácil pero difícil el mantenimiento .	La comunicación puede ser un problema en los sistemas distribuidos, pero es fácil mantener y depurar los pequeños módulos de microservicios .
Orientación a Negocio.	Son menos orientadas a negocio , pero aún se siguen utilizando en gran medida en modelos de negocio.	Se orientan más a los negocios, ya que se basan en SOA que se diseñan específicamente para un sistema empresarial.
Combinación con servicios externos.	La aplicación está fuertemente empaquetada y es una sola entidad , por lo que cada servicio externo debe interactuar con la aplicación completa.	Las aplicaciones y servicios externos pueden interactuar y conectarse fácilmente a cada módulo individual .

CUANDO NO USAR MSA (microservicios)?

1. Equipos Pequeños.

- Si el tamaño del equipo es insuficiente para gestionar cargas de trabajo de microservicios, se producirán retrasos en la entrega.
- Equipos pequeños pueden enfrentar dificultades significativas en la implementación y administración de microservicios.

2. Nuevos Productos.

- No es recomendable iniciar con microservicios en productos nuevos que requieren cambios sustanciales durante el desarrollo.
- Los microservicios pueden ser más costosos al rediseñar dominios de negocio, incluso si la escalabilidad se vuelve crítica.

4. Rediseño de Dominios de Negocio.

- Los microservicios pueden resultar costosos y complejos al rediseñar dominios de negocio, lo que puede ser problemático incluso en escenarios de alta escalabilidad.

5. Antipatrón de Base de Datos Compartida.

- Es crucial evitar la adopción del antipatrón de base de datos compartida, ya que socava los principios de los microservicios y añade complejidades que obstaculizan su eficacia.

Service-Based Architecture (Arquitectura basada en Servicios SBA).

La **Arquitectura basada en Servicios (SBA)** es un enfoque híbrido que combina elementos de microservicios y SOA. Aquí tienes un resumen de sus características principales:

- **Flexibilidad y Menor Complejidad:** Al ser distribuida, resulta menos compleja y costosa que otras arquitecturas, lo que la hace ideal para aplicaciones de negocio.
- **División Funcional:** La funcionalidad se divide en servicios basados en dominios del negocio, permitiendo que cada servicio opere de forma independiente.
- **Compartición de Recursos:** Los servicios pueden compartir bases de datos y la interfaz de usuario.
- **Escalabilidad:** Los servicios que requieren mayor demanda o recursos pueden duplicarse para mejorar el rendimiento.

Variantes de arquitectura en sistemas de microservicios:

- **Interfaz de Usuario:**
 - **Monolítica:** Una interfaz compartida entre todos los servicios.
 - **Basada en dominios:** La interfaz se separa según el dominio de cada servicio.
 - **Basada en servicios:** Cada servicio tiene su propia interfaz.
- **Base de Datos:**
 - **Monolítica:** Una base de datos compartida entre todos los servicios.
 - **Orientada a dominios:** Bases de datos asignadas a cada dominio.
 - **Individuales:** Cada servicio tiene su propia base de datos.
- **API Intermedia:** Una capa de API de Proxy Inverso que enruta solicitudes y puede incluir middlewares de seguridad, métricas, etc.

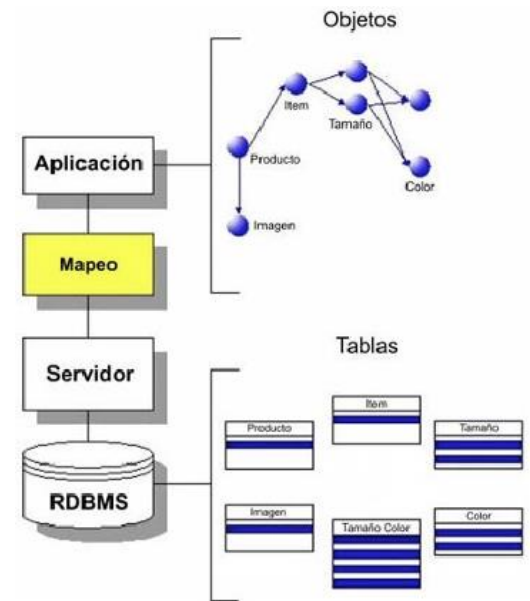
Esta variante agrega una capa API de Proxy Inverso o puerta de enlace, que enruta las solicitudes desde la interfaz de usuario a cada servicio correspondiente. Posibilita exponer la funcionalidad de cada dominio a sistemas externos, y también mover algunos aspectos de la

interfaz fuera de esa capa, como pueden ser middlewares de seguridad, métricas, registros de auditoría, etc.)

DISEÑO DE PERSISTENCIA (chequear).

Persistencia en un sistema de información **se refiere a la capacidad de los datos para sobrevivir a la terminación del proceso que los creó**. Es decir, los datos persisten incluso después de que el programa que los generó haya dejado de ejecutarse, **permitiendo que sean recuperados y utilizados en ejecuciones futuras**. La persistencia se logra generalmente mediante el **almacenamiento de datos en bases de datos**.

En el ámbito de la programación orientada a objetos (POO), uno de los desafíos significativos es el **problema de impedancia** entre el modelo de objetos y el modelo relacional de las bases de datos (RDBMS). Este problema surge porque los conceptos de POO, como herencia, encapsulamiento y asociaciones, no se mapean directamente a las estructuras de datos relacionales, lo que **complica la persistencia de los objetos**.



Para abordar este problema, se hace necesario el proceso de **materialización** (convertir registros de una base de datos en objetos) y **desmaterialización** (convertir objetos en registros de base de datos). Este proceso es esencial para mantener la integridad de los datos y asegurar que la lógica de negocio en los sistemas de información pueda interactuar eficazmente con los datos almacenados.

- ✓ **Estructura y Comportamiento:** Define cómo los objetos y sus atributos se transforman en tablas y columnas en una base de datos relacional.
- ✓ **Mapeo de Clases a Tablas:** Incluye asignar tablas para clases, transformar atributos en columnas y representar instancias de clases como filas en tablas.
- ✓ **Relaciones entre Clases:** Describe cómo se gestionan las asociaciones, agregaciones, composiciones y generalizaciones entre clases en la base de datos.
- ✓ **Reglas de Integridad:** Asegura que los datos sean consistentes y válidos, incluyendo integridad relacional y referencial.

1. Paradigmas de Diseño de Persistencia

Estructurado: Basado en el uso de modelos entidad-relación (ER) y bases de datos relacionales (RDBMS). Se enfoca en la estructura de datos y las relaciones entre ellos.

Orientado a Objetos: Utiliza conceptos de programación orientada a objetos (OOP) para la persistencia, incluyendo clases y objetos.

2. Modelo Entidad-Relación

- **Entidades y Relaciones:** Define cómo se relacionan los datos entre sí.
- **Cardinalidad y Opcionalidad:** Indican la cantidad de instancias de una entidad que se relacionan con otra entidad y si esa relación es opcional u obligatoria.

- **Reglas de Integridad:** Incluyen la integridad relacional (no permitir valores nulos en atributos identificadores) e integridad referencial (asegurar que los valores referenciales existan en la entidad objetivo).

3. Problema de Impedancia

Transformación de Objetos a Tablas: Necesidad de convertir estructuras de datos de objetos a estructuras orientadas a tablas, lo cual puede crear un acoplamiento fuerte entre la aplicación y el DBMS.

4. Mapeo de Clases a Bases de Datos Relacionales

- **Asignar Tablas a Clases:** Cada clase se mapea a una tabla.
- **Transformación de Atributos:** Los atributos de una clase se convierten en columnas de la tabla.
- **Identificadores Únicos:** Uso de claves primarias para identificar de manera única cada instancia de una clase en la tabla.
- **Relaciones:** Las asociaciones con cardinalidad mayor a uno se transforman en nuevas tablas que conectan las tablas originales.

5. Relaciones entre Clases

- **Asociación:** Especifica que los objetos de un elemento se conectan a los objetos de otro.
- **Agregación y Composición:** Tipos especiales de asociaciones donde "todo" y "parte" tienen relaciones de pertenencia.

6. Opciones para el Mapeo de Herencia

- **Eliminar Hijos/Padres:** Opciones para manejar la herencia en el modelo relacional.
- **Simulación:** Transformar la herencia de objetos a un modelo relacional sin eliminar datos.

7. Framework de Persistencia

- **Funciones:** Almacenar y recuperar objetos, gestionar transacciones (commit y rollback).
- **Materialización y Desmaterialización:** Convertir registros en objetos y viceversa.
- **Caché:** Almacén de objetos materializados para mejorar el rendimiento.
- **Proxies Virtuales:** Referencias inteligentes para materialización perezosa (bajo demanda).

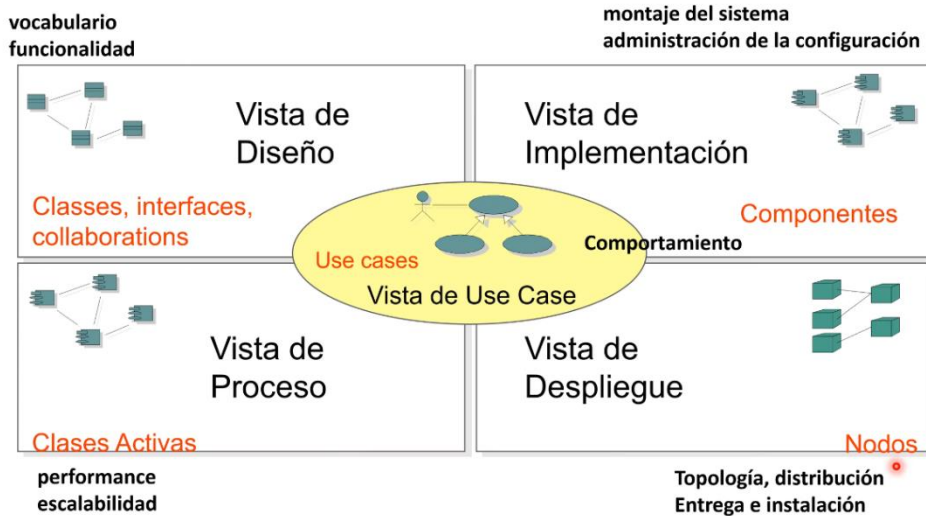
8. Beneficios de Usar ODBMS (Sistemas de Gestión de Bases de Datos Orientadas a Objetos)

- **Almacenamiento Directo de Objetos:** No requiere conversión entre objetos y DBMS.
- **Integración con Lenguajes OOP:** Permite usar el mismo lenguaje de la aplicación para manipular la base de datos.

9. Características de las BDOO

- **Objetos Complejos:** Soporte para objetos complejos.
- **Identidad de Objetos:** Identificación única independiente de los valores internos.
- **Encapsulamiento y Jerarquía:** Soporte para encapsulamiento y herencia.

Vistas en el PUD



VISTAS ARQUITECTONICAS.

VISTA.

Una **proyección de un modelo** (o modelos), el cual es **visto de una perspectiva** o punto de ventaja particular, que **omite entidades que no son relevantes para esta perspectiva**.

MODELO.

Una **descripción completa de un sistema desde una perspectiva particular** y a un nivel de abstracción específico.

PUNTO DE VISTA → Una definición (o descripción) de una vista Contenido, significado y representación (lenguajes, técnicas de modelado).

La **arquitectura 4+1 de Philippe Krutchen** es un enfoque para describir y comprender la arquitectura de un sistema utilizando UML (Lenguaje Unificado de Modelado). Este enfoque se basa en cinco vistas diferentes:

1. **Vista de diseño (lógica):** Se enfoca en la funcionalidad del sistema desde la perspectiva de los desarrolladores. Describe los componentes y sus relaciones.
2. **Vista de implementación:** Muestra la organización del software en módulos y componentes, útil para los desarrolladores.
3. **Vista de procesos:** Se centra en los aspectos dinámicos del sistema, como la concurrencia y la sincronización.
4. **Vista de despliegue:** Describe la topología del hardware y cómo los componentes del software se distribuyen en él.
5. **Vista de casos de uso:** Une las otras cuatro vistas y se enfoca en los requisitos del sistema desde la perspectiva del usuario final.

Vista Arquitectónica de la Funcionalidad (CU).

Esta vista se centra en los **Casos de Uso (CU)** que representan el comportamiento del sistema desde la perspectiva de los actores, como los usuarios finales, analistas y probadores. En otras palabras, describe cómo el sistema interactúa con sus usuarios y cómo se percibe desde el exterior.

- **Importancia:** Es la primera vista que se desarrolla al elegir los casos de uso más significativos para la arquitectura. A partir de esta vista, se construyen las demás vistas arquitectónicas.

- **Cobertura:** Por lo general, esta vista abarca entre el 10% y el 15% de todos los casos de uso del sistema. No necesita incluir todos los casos de uso, pero debe ser representativa para mostrar las decisiones arquitectónicas tomadas en el producto.
- **Enfoque:** La Vista Arquitectónica de la Funcionalidad permite describir los diferentes tipos de funcionalidades del sistema y cómo se relacionan con los actores. Es fundamental para comprender cómo se implementan los requisitos funcionales.

Vista de Diseño (lógica).

Subsistemas y Componentes: En esta vista, identificamos los **subsistemas** del sistema y, dentro de cada uno de ellos, los **componentes** (que son conjuntos de clases). Estos componentes son significativos para la arquitectura y ofrecen servicios a través de interfaces (tanto provistas como requeridas).

- **Elementos Relevantes:** La Vista de Diseño (lógica) nos permite observar cuáles son los elementos relevantes de la arquitectura que el sistema va a manejar. Estos elementos son cruciales para resolver la funcionalidad marcada en la Vista Arquitectónica de la Funcionalidad (CU). Es esencial para traducir los requisitos funcionales en una estructura de software coherente y manejable.
- **Clases, Interfaces y Colaboraciones:** En esta vista, se definen las clases y las interfaces que forman el vocabulario del problema y su solución. Las colaboraciones entre estos elementos también se consideran para garantizar una implementación coherente.
- **Comunicación entre Subsistemas:** Los subsistemas se comunican entre sí a través de sus interfaces. Esto asegura que los diferentes componentes trabajen juntos de manera efectiva para lograr los objetivos del sistema.

La Vista de Procesos (Vista de Interacción).

Se centra en describir la **conurrencia**, el flujo de **control** entre las **partes** y los **elementos de comunicación** en una arquitectura **durante** la **ejecución** del sistema.

En otras palabras, esta vista **proporciona una visión detallada** de cómo los **componentes** del sistema **interactúan** entre sí **durante la ejecución**, especialmente en términos de concurrencia y sincronización

- **Concurrencia:** Esta vista aborda cómo los diferentes componentes o subsistemas interactúan simultáneamente en tiempo de operación. Es especialmente relevante cuando se quieren describir características no funcionales relacionadas con el rendimiento y la disponibilidad.
- **Componentes Multihilo:** En esta vista, se consideran los componentes que funcionan en paralelo, es decir, aquellos que pueden ejecutar múltiples hilos de ejecución al mismo tiempo. Esto es importante para sistemas que requieren alta capacidad de procesamiento o que deben manejar múltiples solicitudes concurrentes.
- **Mecanismos de Concurrencia y Sincronización:** La Vista de Procesos también incluye los mecanismos utilizados para coordinar la ejecución concurrente, como bloqueos, semáforos,

monitores, etc. Estos mecanismos garantizan que los hilos de ejecución se sincronicen correctamente y evitan problemas como condiciones de carrera.

La Vista de Implementación (componentes).

Crucial para estructurar y poner en ejecución el sistema físico.

- **Artefactos y Componentes:** En esta vista, identificamos los **artefactos** (como componentes y módulos de implementación) que se utilizan para ensamblar el sistema. Estos artefactos incluyen código fuente, bibliotecas y otros elementos necesarios para la implementación.
- **Correspondencia:** La vista también establece la correspondencia entre los **elementos de software** (como clases y componentes lógicos) y los artefactos físicos. Es decir, cómo se traducen los componentes lógicos en implementaciones concretas.
- **Propósito:** La Vista de Implementación toma decisiones sobre cómo estructurar el repositorio donde se almacenarán los componentes de código. También aborda el control de cambios, la configuración y las políticas de gestión de configuración. En resumen, es fundamental para la gestión efectiva del desarrollo y la implementación del software.

La Vista de Despliegue.

Se enfoca en la distribución física del sistema y cómo se ejecuta en la infraestructura de hardware (HW). Es fundamental para comprender cómo se implementa el sistema en la infraestructura física y cómo se distribuyen los componentes de software en los nodos de hardware.

- **Topología de Hardware:** Describe cómo los nodos de hardware (como computadoras y dispositivos) se organizan para formar la infraestructura sobre la cual se ejecuta el sistema. Esto incluye la disposición de servidores, estaciones de trabajo, dispositivos móviles, etc.
- **Niveles de Hardware:** La vista muestra los diferentes niveles de hardware involucrados. Por ejemplo, puede haber servidores de aplicación, servidores de base de datos, dispositivos de red, etc. Cada nivel tiene su función específica en el sistema.
- **Componentes de Software:** También se considera cómo los componentes de software (como módulos, servicios o aplicaciones) se alojan en los nodos de hardware. Por ejemplo, qué parte del sistema se ejecuta en qué servidor o dispositivo.
- **Distribución y Comunicación:** La Vista de Despliegue se ocupa de la distribución, entrega e instalación de las partes físicas del sistema. Además, muestra cómo se comunican los diferentes niveles de hardware entre sí y con los componentes de software.

PRINCIPIOS DE DISEÑO DE SOFTWARE ORIENTADO A OBJETOS.

DRY → No te repitas, evitar duplicaciones de código y crear más abstracciones

SOC → Separación de intereses, aislar la aplicación de software en secciones separadas. Cada sección debe abordar una preocupación distinta que tenga mínima superposición con las demás.

- Gestiona la complejidad y aumenta la mantenibilidad.
- Mejora la cohesión y reduce el acoplamiento
- Facilita la extensión OCP

- Permite la reutilización
Reduce la fragilidad de los módulos

YAGNI → No lo vas a necesitar, recalcando la pérdida de tiempo que trae crear código para usarlo en un futuro.

KISS → Keep it simple, stupid. Mayor calidad de código simple y fácil de mantener y mejorar.
Implicaciones de este principio.

- No aumentar infinitamente el nivel de abstracción → incrementa mucho la complejidad.
- No incluir funciones que no se usen (YAGNI)
- Que sentido tiene incluir librerías de gran tamaño si al fin y al cabo se usa solo una función de ella.
- Se recomienda descomponer la complejidad en partes más simples, sosteniéndose sobre el principio DRY.

Ley de Demeter/Ppio de menor conocimiento → producir objetos que estén débilmente acoplados. Los objetos fuertemente acoplados dependen en gran medida unos de otros y requieren una gran cantidad de información entre sí para que los métodos funcionen.

Los objetos débilmente acoplados tienen un número mínimo de dependencias y saben poco sobre los métodos de otras clases (DONT TALK TO STRANGERS).

Tell, Dont ask → solicitar a los objetos que hagan cosas, un objeto está definido por su comportamiento es por ello por lo que es esencial su distribución de responsabilidades (POLIMORFISMO).

Program to interface principle → refiere a la práctica de diseñar sistemas desacoplados y flexibles.

- Usar interfaces para desacoplar el sistema propio de un sistema externo (boundary)
- Para múltiples implementaciones concretas se utiliza una interfaz que muestre todo.
- Facilita la extensión y modificación del código sin afectar otras partes del sistema.

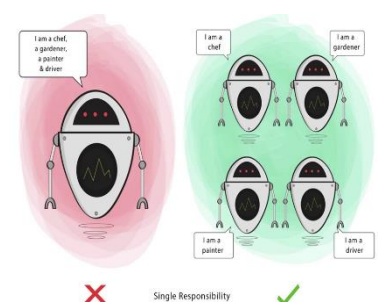
Herencia – mecanismo de reutilización a nivel de clases, heredando atributos/comportamientos de otra, jerarquía de clases.

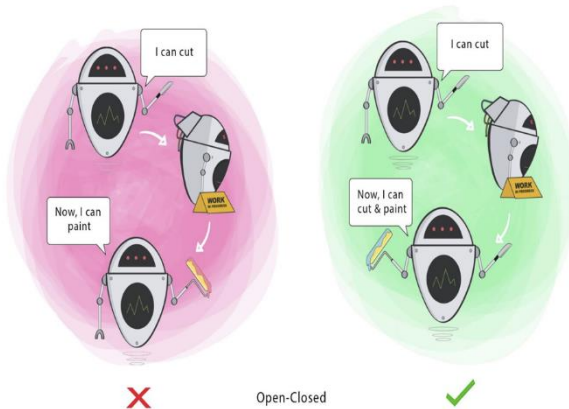
Composición – mecanismo de reutilización a nivel de objetos, construcción de objetos complejos a partir de objetos mas simples. La composición es preferida sobre la herencia cuando se busca flexibilidad y mantenibilidad, ya que permite cambiar el comportamiento de un objeto en tiempo de ejecución.

Delegación – principio de diseño en la programación orientada a objetos que permite reutilizar funcionalidades de otras clases sin cambiar su implementación (**COMPOSITE REUSE PRINCIPLE**).

PRINCIPIOS SOLID.

Single Responsibility Principle (SRP): Cada clase debe tener una **única responsabilidad** o motivo para cambiar. Promueve alta cohesión.



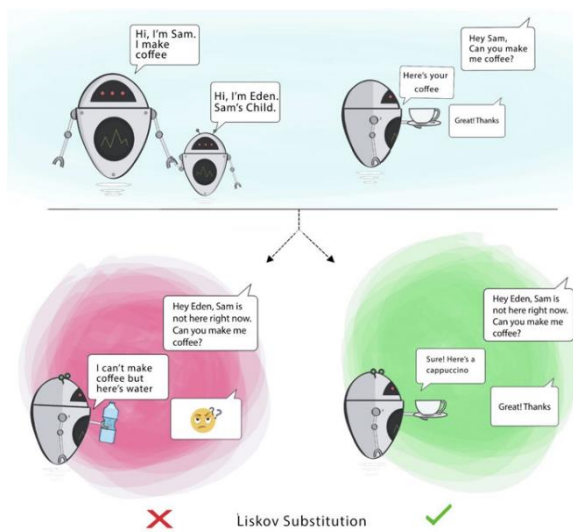


Open/Closed Principle (OCP):

Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación.

Cerrado para agregar más métodos (para mantener SRP) pero abierto a usar mediante interfaces (extensión) otras clases para cumplir cierta funcionalidad. **[Permitir el cambio + No modificar lo existente = Flexibilidad]**

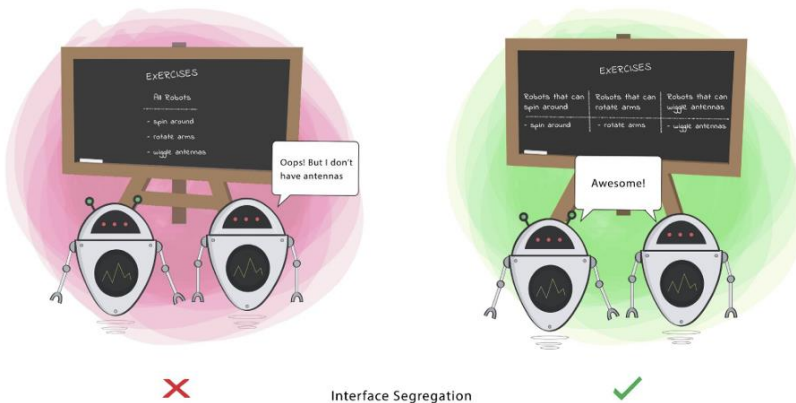
Liskov Substitution Principle (LSP): Los objetos de una clase hija deben poder sustituir a los objetos de la clase padre sin alterar el funcionamiento del programa. Asegura que una clase hija pueda reemplazar a su clase padre sin que el comportamiento del programa se vea afectado negativamente.



Un **ejemplo** común para ilustrar el **LSP** es el "problema del rectángulo y el cuadrado". Si tenemos una clase `Rectángulo` con propiedades `ancho` y `alto`, y luego creamos una clase `Cuadrado` que hereda de `Rectángulo`, debemos asegurarnos de que las propiedades `ancho` y `alto` se comporten correctamente en ambas clases. Si modificamos el comportamiento en `Cuadrado` para que `ancho` y `alto` siempre sean iguales (ya que un cuadrado tiene lados iguales), estaríamos violando el LSP, ya que no podríamos sustituir un `Cuadrado` por un `Rectángulo` sin cambiar el resultado del programa.

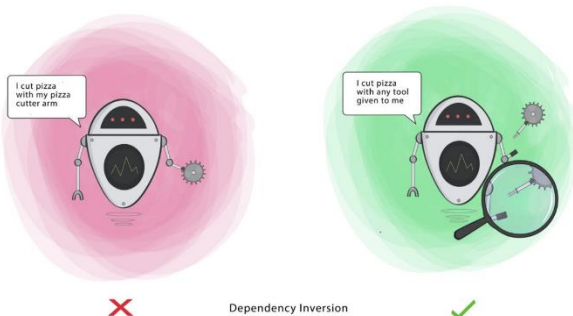
Interface Segregation Principle (ISP):

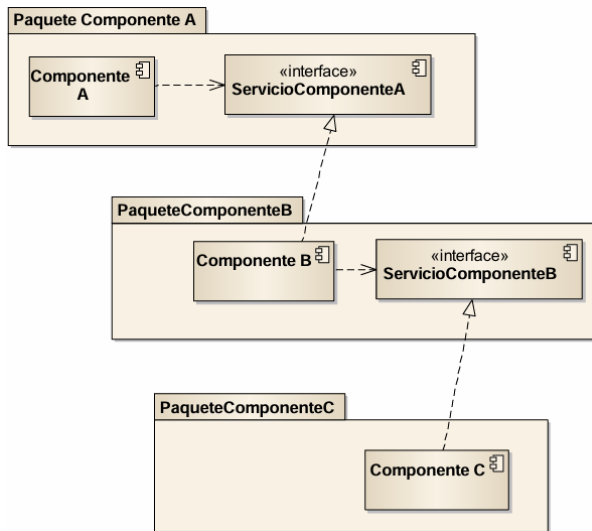
Los clientes no deben verse obligados a depender de interfaces que no utilizan. Las **interfaces deben ser específicas** y contener solo los métodos necesarios para evitar que las clases implementen métodos innecesarios.



Dependency Inversion Principle (DIP):

Módulos de alto y bajo nivel: Los módulos de alto nivel no deben depender de los módulos de bajo nivel, ambos deben depender de abstracciones.





Abstracciones sobre detalles: Las abstracciones no deben depender de los detalles; son los detalles los que deben depender de las abstracciones (**INTERFACES**).

Flexibilidad y reutilización: se logra una mayor flexibilidad y robustez en el sistema. Los cambios en los módulos de bajo nivel no afectarán directamente a los módulos de alto nivel, ya que ambos se comunican a través de **interfaces abstractas**.

La inversión de dependencias permite que los **módulos** sean más móviles y reutilizables. Pueden **ser reemplazados o actualizados sin afectar la estructura general del sistema**.

En lugar de que los módulos de alto nivel dependan directamente de los módulos de bajo nivel, ambos deben depender de abstracciones. **Esto significa que los detalles específicos no deben ser la base de las relaciones entre los módulos.**